MVP2 Mobile Apps Specification

MVP2 Mobile Apps Specification

Revision: 3 **Last modified:** 2026-07-04T16:30:00Z

> **Revision 3 changelog:** added the missing Mermaid state diagram for the Android foreground-service lifecycle next to §2.10 (the section this diagram synthesizes: §2.2 VpnService start/stop, §2.3 foreground notification, §2.4 Doze/battery-optimization handling, and this section's own Android 15+ UIDT keepalive fallback); corrected §5.1's Flutter `HelixTheme` brand-color seed values, which hardcoded an independent indigo (`#6366F1`) never reconciled against the canonical, already-built OpenDesign token system (`docs/research/CROSS_CUTTING_GAP_ANALYSIS.md` §1.2 finding #1).

Helix VPN Mobile Client — Android, iOS, and HarmonyOS

Version: 1.1.0
Date: July 2025 (Revision 2: 2026-07-04)
Status: Draft for Implementation
Related Documents: MVP2 Architecture Spec, MVP2 Shared Core Spec, MVP2 Desktop Spec

> **Revision 2 changelog:** reconciled the protocol list (WireGuard, Shadowsocks, MASQUE, Multi-Hop; `OpenVPN` is a RESERVED/unimplemented enum placeholder only — `MVP2_SHARED_CORE.md` §2.3) across all three architecture diagrams and the server protocol-badge widget; corrected the FFI-bridge description (UniFFI is used specifically for the iOS Swift Network Extension interop layer, not for Android); corrected code-reuse percentages (72% / 72% / 70%) and the cross-platform bundle/RAM budgets (<25MB / <120MB idle) to match `MVP2_ARCHITECTURE.md` §4; reconciled

> every platform's connection-state enum (Kotlin `VpnConnectionState`, Swift `TunnelState`, ArkTS `TunnelState`, Dart `ConnectionState`) to the single canonical `Disconnected → Connecting → Connected → Reconnecting → KillSwitchActive → Disconnecting → (Disconnected)` state machine owned by `helix-vpn-engine` (`MVP2_ARCHITECTURE.md` §5.6) — no platform-invented states remain; corrected the CI pipeline's Flutter version pinning (Android/iOS use Flutter 3.29+, only the HarmonyOS job uses the community `3.22.0-ohos` embedding); wired `PlatformAdapter::apply_managed_policy` through each platform's managed-configuration channel; added a new §8 Enterprise Hardening & Production Readiness section (app-store review, MDM/managed-config, staged rollout + rollback, crash reporting, telemetry consent, offline/degraded-network behavior, accessibility, localization, biometric fallback, multi-account, enterprise SSO, license/entitlement checks); added 3 Mermaid diagrams (this document previously had none).

Table of Contents

1. Mobile Client Overview
2. Android Client
3. iOS Client
4. HarmonyOS Client
5. Mobile UI/UX Design
6. Mobile-Specific Features
7. Build & Distribution
8. Enterprise Hardening & Production Readiness

---

1. Mobile Client Overview

1.1 Unified Mobile Strategy

The Helix VPN mobile strategy employs a **unified Flutter codebase** (Flutter 3.29+ with the Impeller renderer on Android/iOS; the community `ohos` embedding pinned at `3.22.0-ohos` on HarmonyOS — see §7.4 for why the CI pipeline pins two different Flutter versions) with platform-specific native integrations for VPN tunnel management. Per the reconciled cross-platform code-reusability analysis (`MVP2_ARCHITECTURE.md` §4), this achieves **72% code reuse on Android, 72% on iOS, and 70% on HarmonyOS** — not the "85-90%" figure quoted in an earlier draft of this document, which double-counted UI-only reuse without netting out the native VPN-adapter layer. Target bundle size is **<25MB** and idle RAM is **<120MB** on all three platforms (`MVP2_ARCHITECTURE.md` §4; see Appendix B for the full per-platform matrix).

```
┌──────────────────────────────────────────────────────────────┐
│                    Flutter UI Layer (Dart)                    │
│  ┌────────────┐ ┌──────────────┐ ┌────────────────────────┐  │
│  │ Connection │ │   Settings   │ │  Server Selection      │  │
│  │   Screen   │ │   Screen     │ │   Screen               │  │
│  └────────────┘ └──────────────┘ └────────────────────────┘  │
│  ┌────────────┐ ┌──────────────┐ ┌────────────────────────┐  │
│  │ Split Tunnel│ │ Account/     │ │  Stats & Analytics     │  │
│  │ Config     │ │ Profile      │ │                        │  │
│  └────────────┘ └──────────────┘ └────────────────────────┘  │
├──────────────────────────────────────────────────────────────┤
│              flutter_rust_bridge v2+ (Dart FFI)                │
├──────────────────────────────────────────────────────────────┤
│                    Shared Rust Core (helix-core)              │
│  ┌─────────────┐ ┌──────────────┐ ┌───────────────────────┐ │
│  │ WireGuard   │ │ Shadowsocks  │ │ MASQUE (RFC 9298) /   │ │
│  │ (primary)   │ │ SIP022       │ │ Multi-Hop             │ │
│  └─────────────┘ └──────────────┘ └───────────────────────┘ │
│  ┌─────────────┐ ┌──────────────┐ ┌───────────────────────┐ │
│  │ Encryption  │ │ Connection   │ │ DNS Leak Protection   │ │
│  │ (ChaCha20)  │ │ State Mgmt   │ │                       │ │
│  └─────────────┘ └──────────────┘ └───────────────────────┘ │
├──────────────────┬──────────────────┬────────────────────────┤
│   Android        │       iOS        │    HarmonyOS            │
│  ┌────────────┐  │  ┌────────────┐  │  ┌─────────────┐       │
│  │ VpnService │  │  │ NEPacket   │  │  │ VpnExtension│       │
│  │ (Kotlin)   │  │  │ TunnelProv │  │  │ Ability     │       │
│  │            │  │  │ (Swift)    │  │  │ (ArkTS)     │       │
│  └────────────┘  │  └────────────┘  │  └─────────────┘       │
│  ┌────────────┐  │  ┌────────────┐  │  ┌─────────────┐       │
│  │ JNI Bridge │  │  │ FRB (Dart) │  │  │ N-API Bridge│       │
│  │ (FRB) to   │  │  │ + UniFFI   │  │  │ to Rust     │       │
│  │ Rust       │  │  │ (NE Swift) │  │  │             │       │
│  └────────────┘  │  └────────────┘  │  └─────────────┘       │
└──────────────────┴──────────────────┴────────────────────────┘
```

Note (OpenVPN): `helix-core`'s `ProtocolType` enum carries a RESERVED `OpenVPN` variant for a possible post-MVP2 addition. There is no `helix-openvpn` crate and no OpenVPN entry in the supported-protocol table; selecting it returns `HelixError::UnsupportedProtocol` in MVP2 builds. No mobile UI, server-list badge, or protocol picker in this document may present OpenVPN as an available option — this reconciles several places in this document (§2.1, §4.1, §5.4) that referenced OpenVPN as if it shipped.

1.2 Platform-Specific Adaptations

Aspect	Android	iOS	HarmonyOS
VPN API	`VpnService` + `Builder`	`NEPacketTunnelProvider`	`VpnExtensionAbility`
Native Language	Kotlin	Swift	ArkTS + C++ (N-API)
Bridge to Rust	JNI via flutter_rust_bridge (FRB only — no UniFFI)	flutter_rust_bridge (Dart↔Rust, main app) **+** UniFFI (Swift↔Rust, Network Extension only)	N-API via flutter_rust_bridge
Background Model	Foreground Service	System-managed extension	ExtensionAbility lifecycle
Credential Store	Android Keystore	iOS Keychain	HUKS (Huawei Universal Keystore)
Push Notifications	FCM (Firebase)	APNs	HMS Push Kit
Min OS Version	Android 8.0 (API 26) — product-supported minimum	iOS 15.0	HarmonyOS NEXT (API 12)

> **FFI clarification:** Android uses `flutter_rust_bridge` (FRB) exclusively for its Dart↔Rust bridge — there is no UniFFI involvement on Android. iOS uses FRB for the main app's Dart↔Rust calls, **plus** UniFFI specifically to bridge the separate `PacketTunnelProvider` Network Extension process's Swift code to `helix-core`, because the NE runs outside the Flutter engine and cannot use FRB's Dart-side codegen. An earlier draft of §1.4 incorrectly described UniFFI as generating "Kotlin and Swift" bindings; UniFFI applies to the iOS Swift NE layer only.

1.3 System Requirements

Android:

- Minimum SDK: **API 26 (Android 8.0 Oreo)** — this is the product-supported minimum (matches `MVP2_SHARED_CORE.md` §5.1). It is also the build-toolchain floor: the Gradle `minSdk` (§2.7) and the `cargo-ndk` Rust cross-compilation target both target API 26, so there is no lower toolchain floor to distinguish from the product minimum on this platform. Do not lower `minSdk` to chase a wider device footprint without a corresponding constitutional product-scope decision — API 26

is required for the modern `VpnService` route/DNS APIs this spec depends on (§2.2).
- Target SDK: API 35 (Android 15)
- Architecture: arm64-v8a, armeabi-v7a, x86_64 (for emulator)
- Permissions: `BIND_VPN_SERVICE`, `FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_SPECIAL_USE`, `FOREGROUND_SERVICE_DATA_SYNC`, `POST_NOTIFICATIONS`, `ACCESS_NETWORK_STATE`, `INTERNET`

iOS:

- Minimum Deployment Target: iOS 15.0
- Architectures: arm64 (devices), arm64-sim (simulator where supported)
- Entitlements: `com.apple.developer.networking.networkextension` (packet-tunnel-provider)
- Capabilities: Network Extensions, App Groups, Keychain Sharing
- Physical device required for VPN testing (simulator does not support Network Extensions)

HarmonyOS:

- Minimum API: 12 (HarmonyOS NEXT)
- Architecture: arm64-v8a
- Permissions: `ohos.permission.MANAGE_VPN`, `ohos.permission.INTERNET`, `ohos.permission.GET_NETWORK_INFO`
- ACL required for `MANAGE_VPN` permission (system_grant)

1.4 Architecture Principles

1. **Rust-First Core:** All VPN protocol logic, encryption, and state management lives in the shared `helix-core` Rust library. Native platform code is a thin adapter layer.
2. **Memory-Conscious iOS:** The iOS NEPacketTunnelProvider operates under a strict ~15MB memory limit. The Swift wrapper must be minimal; all heavy lifting is done in optimized Rust code.
3. **Foreground Service for Android:** Android requires a persistent foreground service with notification. The service must handle Doze mode, manufacturer-specific battery optimizations, and Android 15's 6-hour foreground service limit.
4. **UniFFI for the iOS Network Extension Only:** Rust↔Swift bindings for the standalone `PacketTunnelProvider` process (which cannot use `flutter_rust_bridge`'s Dart-side codegen because it never loads the Flutter engine) are auto-generated using Mozilla's UniFFI, reducing hand-written binding code and potential memory safety issues. Android has no UniFFI dependency — its JNI bridge is generated entirely by `flutter_rust_bridge`.
5. **Enterprise-Managed Policy is Adapter-Level:** All three platforms implement

`PlatformAdapter::apply_managed_policy(policy: ManagedPolicy) -> Result<()>` (`MVP2_SHARED_CORE.md` §3.1) so an MDM/EMM-pushed policy (allowed protocols, forced kill switch, forced split-tunnel, forced DNS, require-SSO) is applied through the same code path as a user-initiated settings change — see §8.2.

---

2. Android Client

2.1 Architecture

The Android client uses a three-layer architecture:

```
┌─────────────────────────────────────────┐
│         Flutter UI Layer (Dart)          │
│  - Connection screen, settings, stats    │
│  - MethodChannel for native calls        │
├─────────────────────────────────────────┤
│      flutter_rust_bridge Layer           │
│  - Auto-generated Dart ↔ Rust bindings   │
│  - Handles memory safety at FFI boundary │
├─────────────────────────────────────────┤
│      Native Platform Layer (Kotlin)      │
│  - HelixVpnService extends VpnService    │
│  - MethodChannel handler for Flutter     │
│  - Foreground service management         │
│  - Notification builder                  │
│  - Quick Settings TileService            │
├─────────────────────────────────────────┤
│         Shared Rust Core                 │
│  - helix-core via JNI (flutter_rust_bridge, no UniFFI) │
│  - WireGuard (primary) / Shadowsocks /   │
│    MASQUE / Multi-Hop protocol impl      │
│    (OpenVPN: reserved, unimplemented)    │
│  - Encryption & packet routing           │
└─────────────────────────────────────────┘
```

2.2 VpnService Implementation

2.2.1 Builder Pattern for Interface Configuration

The `HelixVpnService` extends Android's `VpnService` and uses the `Builder` inner class to configure the virtual network interface:

```
/**
 * Mirrors the single canonical `ConnectionStatus` state machine owned by
 * `helix-vpn-engine` (`MVP2_ARCHITECTURE.md` §5.6). Android reports
```

```kotlin
 * low-level events (`onRevoke`, permission changes, network
       callbacks) up to
 * the Rust core via [HelixRustBridge]; the core — never this
       enum's
 * assignment sites directly — decides the next state. This
       service only
 * *mirrors* the resulting state for `MethodChannel` / Tile /
       notification
 * consumers. No Android-specific state (e.g. a bespoke `REVOKED`
       or `ERROR`
 * value) is added here — a system VPN-permission revocation
 * ([onRevoke]) is reported as a transition to [DISCONNECTED] with
       a
 * `disconnectReason` side-channel extra, not as a new enum value.
 */
enum class VpnConnectionState {
    DISCONNECTED,
    CONNECTING,
    CONNECTED,
    RECONNECTING,
    KILL_SWITCH_ACTIVE,
    DISCONNECTING,
    CONNECTION_FAILED,
}

class HelixVpnService : VpnService() {

    companion object {
        const val ACTION_CONNECT = "com.helix.vpn.CONNECT"
        const val ACTION_DISCONNECT = "com.helix.vpn.DISCONNECT"
        const val NOTIFICATION_ID = 1
        const val CHANNEL_ID = "helix_vpn_channel"

        @JvmStatic
        var isRunning = false
            private set

        @JvmStatic
        var connectionState: VpnConnectionState =
        VpnConnectionState.DISCONNECTED
            private set

        @JvmStatic
        var lastDisconnectReason: String? = null
            private set
    }

    private var vpnInterface: ParcelFileDescriptor? = null
    private var tunnelThread: Thread? = null
    private val rustBridge = HelixRustBridge() // JNI wrapper for
        helix-core
```

```kotlin
override fun onStartCommand(intent: Intent?, flags: Int,
    startId: Int): Int {
    when (intent?.action) {
        ACTION_CONNECT -> {
            val config =
    intent.getParcelableExtra<VpnConfig>("config")
                ?: return START_NOT_STICKY
            startVpnConnection(config)
        }
        ACTION_DISCONNECT -> {
            stopVpnConnection()
        }
        else -> {
            // Service restarted by system (always-on VPN)
            val savedConfig = loadSavedConfiguration()
            if (savedConfig != null) {
                startVpnConnection(savedConfig)
            }
        }
    }
    return START_STICKY // Restart if killed by system
}

private fun startVpnConnection(config: VpnConfig) {
    // Build VPN interface configuration
    val builder = Builder().apply {
        setSession("Helix VPN")
        setMtu(config.mtu) // Typically 1420 for WireGuard

        // Configure tunnel IP address
        addAddress(config.tunnelAddress,
    config.tunnelPrefixLength)

        // Configure DNS servers (critical for leak
    prevention)
        config.dnsServers.forEach { dns ->
            addDnsServer(dns)
        }

        // Add search domains
        config.searchDomains.forEach { domain ->
            addSearchDomain(domain)
        }

        // Route configuration
        if (config.routeAllTraffic) {
            addRoute("0.0.0.0", 0)  // IPv4 default route
            addRoute("::", 0)        // IPv6 default route
        } else {
            config.includedRoutes.forEach { route ->
                addRoute(route.address, route.prefixLength)
            }
```

```
    }

    // Exclude routes (Android 13+ / API 33+)
    if (Build.VERSION.SDK_INT >=
Build.VERSION_CODES.TIRAMISU) {
        config.excludedRoutes.forEach { route ->

excludeRoute(IpPrefix(InetAddress.getByName(route.address),
route.prefixLength))
        }
    }

    // Split tunneling: allowed applications (whitelist
mode)
    if (config.allowedApplications.isNotEmpty()) {
        config.allowedApplications.forEach { packageName
->
            try {
                addAllowedApplication(packageName)
            } catch (e:
PackageManager.NameNotFoundException) {
                Log.w("HelixVPN", "Package not found:
$packageName")
            }
        }
    }

    // Split tunneling: disallowed applications (blacklist
mode)
    // Mutually exclusive with allowedApplications
    if (config.disallowedApplications.isNotEmpty() &&
config.allowedApplications.isEmpty()) {
        config.disallowedApplications.forEach {
packageName ->
            try {
                addDisallowedApplication(packageName)
            } catch (e:
PackageManager.NameNotFoundException) {
                Log.w("HelixVPN", "Package not found:
$packageName")
            }
        }
    }

    // Allow apps to bypass VPN using bindProcessToNetwork
    if (config.allowBypass) {
        allowBypass()
    }

    // HTTP proxy configuration (Android 10+ / API 29+)
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q &&
config.httpProxy != null) {
        val proxyInfo = ProxyInfo.buildDirectProxy(
            config.httpProxy.host,
```

```kotlin
                config.httpProxy.port
            )
            setHttpProxy(proxyInfo)
        }

        // Blocking mode for the TUN file descriptor
        setBlocking(true)
    }

    // Establish the VPN interface (creates TUN device)
    vpnInterface = builder.establish()

    if (vpnInterface != null) {
        // Start foreground service with persistent
    notification
        val notification =
    buildVpnNotification(config.serverLocation)
        if (Build.VERSION.SDK_INT >=
    Build.VERSION_CODES.UPSIDE_DOWN_CAKE) {
            startForeground(NOTIFICATION_ID, notification,

    ServiceInfo.FOREGROUND_SERVICE_TYPE_SPECIAL_USE)
        } else {
            startForeground(NOTIFICATION_ID, notification)
        }

        // Pass TUN fd to Rust core for packet processing
        val tunFd = vpnInterface!!.fd
        rustBridge.startTunnel(tunFd, config.toRustConfig())

        isRunning = true
        connectionState = VpnConnectionState.CONNECTED
        broadcastStateChange(connectionState)
    }
}

private fun stopVpnConnection() {
    rustBridge.stopTunnel()
    vpnInterface?.close()
    vpnInterface = null
    isRunning = false
    connectionState = VpnConnectionState.DISCONNECTED
    broadcastStateChange(connectionState)
    stopForeground(STOP_FOREGROUND_REMOVE)
    stopSelf()
}

override fun onRevoke() {
    // Called when user revokes VPN permission or another VPN
    app takes
    // over. This is NOT a new canonical state — it reports
    into the same
```

```
        // DISCONNECTED state as any other teardown, with the
        reason carried
        // as a side-channel so the UI can show "VPN permission
        revoked"
        // without the engine needing a bespoke `Revoked` enum
        value.
        Log.w("HelixVPN", "VPN permission revoked by system or
        user")
        lastDisconnectReason = "permission_revoked"
        stopVpnConnection()
    }

    override fun onDestroy() {
        stopVpnConnection()
        super.onDestroy()
    }
}
```

2.2.2 Split Tunneling Configuration

Android provides the most comprehensive split tunneling on mobile through two mutually exclusive mechanisms:

Whitelist Mode (allowedApplications):

```
// Only specified apps use the VPN; all other traffic bypasses
builder.addAllowedApplication("com.example.corporate_app")
builder.addAllowedApplication("com.example.secure_browser")
// All other apps connect directly
```

Blacklist Mode (disallowedApplications):

```
// Specified apps bypass the VPN; all other traffic goes through
        VPN
builder.addDisallowedApplication("com.bank.app")        // Banking
        bypasses VPN
builder.addDisallowedApplication("com.netflix.app")     //
        Streaming bypasses VPN
builder.addDisallowedApplication("com.spotify.music")   // Music
        bypasses VPN
```

IP-based Split Tunneling (API 33+):

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
    // Exclude LAN traffic from VPN
    builder.excludeRoute(IpPrefix(InetAddress.getByName("192.168.0.0"), 16))
    builder.excludeRoute(IpPrefix(InetAddress.getByName("10.0.0.0"), 8))
    builder.excludeRoute(IpPrefix(InetAddress.getByName("172.16.0.0"), 12))
}
```

Pre-API 33 Workaround: For devices running Android 12 and below, the app calculates all non-excluded routes using the WireGuard AllowedIPs calculator approach and adds them individually via `builder.addRoute()`.

2.2.3 Always-On VPN and Lockdown Mode

```kotlin
object AlwaysOnVpnHelper {

    /**
     * Check if this VPN is running as an always-on VPN.
     * Returns true if the system auto-started this service.
     */
    fun isAlwaysOn(context: Context): Boolean {
        return if (Build.VERSION.SDK_INT >=
        Build.VERSION_CODES.Q) {
            val vpnService = context as? VpnService
            vpnService?.isAlwaysOn ?: false
        } else {
            false
        }
    }

    /**
     * Check if lockdown mode (kill switch) is active.
     * When lockdown is enabled, all traffic is blocked if VPN
        disconnects.
     */
    fun isLockdownEnabled(context: Context): Boolean {
        return if (Build.VERSION.SDK_INT >=
        Build.VERSION_CODES.Q) {
            val vpnService = context as? VpnService
            vpnService?.isLockdownEnabled ?: false
        } else {
            false
        }
    }

    /**
     * Guide the user to system settings to enable always-on VPN.
     * Apps cannot programmatically enable always-on (requires
        system/MDM).
     */
    fun openVpnSettings(context: Context) {
        val intent = Intent(Settings.ACTION_VPN_SETTINGS)
        context.startActivity(intent)
    }

    /**
     * Metadata in AndroidManifest.xml to declare always-on
        support:
```

```
     * <meta-data
        android:name="android.net.VpnService.SUPPORTS_ALWAYS_ON"
     *             android:value="true" />
     */
}
```

AndroidManifest.xml declarations:

```
<service
    android:name=".vpn.HelixVpnService"
    android:permission="android.permission.BIND_VPN_SERVICE"
    android:foregroundServiceType="specialUse|dataSync"
    android:exported="true">
    <intent-filter>
        <action android:name="android.net.VpnService" />
    </intent-filter>
    <meta-data
        android:name="android.net.VpnService.SUPPORTS_ALWAYS_ON"
        android:value="true" />
    <property
        android:name="android.app.PROPERTY_SPECIAL_USE_FGS_SUBTYPE"
        android:value="vpn" />
</service>
```

2.3 Foreground Service with Persistent Notification

```
class VpnNotificationHelper(private val context: Context) {

    init {
        createNotificationChannel()
    }

    private fun createNotificationChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                HelixVpnService.CHANNEL_ID,
                "Helix VPN Connection",
                NotificationManager.IMPORTANCE_LOW
            ).apply {
                description = "Shows when Helix VPN is active"
                setShowBadge(false)
                enableLights(false)
                enableVibration(false)
                lockscreenVisibility =
        Notification.VISIBILITY_PUBLIC
            }

            val notificationManager =
        context.getSystemService(NotificationManager::class.java)
            notificationManager.createNotificationChannel(channel)
        }
    }
```

```kotlin
fun buildVpnNotification(serverLocation: String, protocol:
    String = "WireGuard"): Notification {
    // Intent to open the app when notification is tapped
    val contentIntent = PendingIntent.getActivity(
        context,
        0,

    context.packageManager.getLaunchIntentForPackage(context.packageName),
        PendingIntent.FLAG_UPDATE_CURRENT or
    PendingIntent.FLAG_IMMUTABLE
    )

    // Intent to disconnect VPN
    val disconnectIntent = PendingIntent.getService(
        context,
        1,
        Intent(context, HelixVpnService::class.java).apply {
            action = HelixVpnService.ACTION_DISCONNECT
        },
        PendingIntent.FLAG_UPDATE_CURRENT or
    PendingIntent.FLAG_IMMUTABLE
    )

    // Intent to pause VPN (temporary disconnect)
    val pauseIntent = PendingIntent.getService(
        context,
        2,
        Intent(context, HelixVpnService::class.java).apply {
            action = "com.helix.vpn.PAUSE"
        },
        PendingIntent.FLAG_UPDATE_CURRENT or
    PendingIntent.FLAG_IMMUTABLE
    )

    return NotificationCompat.Builder(context,
    HelixVpnService.CHANNEL_ID)
        .setContentTitle("Helix VPN Connected")
        .setContentText("Server: $serverLocation \u00B7
    Protocol: $protocol")
        .setSmallIcon(R.drawable.ic_notification_vpn)
        .setContentIntent(contentIntent)
        .setOngoing(true) // Cannot be dismissed by user
        .setPriority(NotificationCompat.PRIORITY_LOW)
        .setCategory(NotificationCompat.CATEGORY_SERVICE)
        .addAction(R.drawable.ic_pause, "Pause", pauseIntent)
        .addAction(R.drawable.ic_disconnect, "Disconnect",
    disconnectIntent)
        .setForegroundServiceBehavior(NotificationCompat.FOREGROUND_SERVICE_IM
        .build()
}
```

```kotlin
    fun updateNotification(serverLocation: String, stats:
        ConnectionStats) {
        val notification = buildVpnNotification(serverLocation)
        val notificationManager =
        context.getSystemService(NotificationManager::class.java)
        notificationManager.notify(HelixVpnService.NOTIFICATION_ID,
        notification)
    }
}
```

2.4 Battery Optimization Handling

```kotlin
object BatteryOptimizationHelper {

    /**
     * Check if the app is whitelisted from battery optimizations.
     * Foreground services should still function without this, but
     * some manufacturers aggressively kill non-whitelisted apps.
     */
    fun isIgnoringBatteryOptimizations(context: Context): Boolean
        {
        val powerManager =
        context.getSystemService(Context.POWER_SERVICE) as
        PowerManager
        return
        powerManager.isIgnoringBatteryOptimizations(context.packageName)
    }

    /**
     * Request battery optimization exemption.
     * NOTE: Google Play restricts use of
        ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS.
     * Play Store apps should guide users to settings instead.
     */
    fun requestBatteryOptimizationExemption(context: Context) {
        // Safe approach for Google Play Store apps:
        val intent =
        Intent(Settings.ACTION_IGNORE_BATTERY_OPTIMIZATION_SETTINGS)
        context.startActivity(intent)
    }

    /**
     * For non-Play Store builds (F-Droid, direct APK):
     * Can directly request exemption.
     */
    fun directRequestExemption(context: Context) {
        if (!isIgnoringBatteryOptimizations(context)) {
            val intent =
        Intent(Settings.ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS).apply
        {
                data = Uri.parse("package:${context.packageName}")
            }
            context.startActivity(intent)
```

```kotlin
        }
    }

    /**
     * Manufacturer-specific battery optimization guides.
     * OEMs like Samsung, Xiaomi, OnePlus have additional
       restrictions.
     */
    fun getManufacturerGuide(): String? {
        return when (Build.MANUFACTURER.lowercase()) {
            "samsung" -> "samsung_battery_guide"
            "xiaomi", "redmi", "poco" -> "xiaomi_battery_guide"
            "oneplus", "oppo" -> "oppo_battery_guide"
            "huawei", "honor" -> "huawei_battery_guide"
            "vivo", "iqoo" -> "vivo_battery_guide"
            else -> null
        }
    }

    /**
     * Monitor Doze mode changes to adjust keepalive intervals.
     */
    fun registerDozeReceiver(context: Context, callback:
        (Boolean) -> Unit) {
        val receiver = object : BroadcastReceiver() {
            override fun onReceive(context: Context, intent:
        Intent) {
                val powerManager =
        context.getSystemService(Context.POWER_SERVICE) as
        PowerManager
                callback(powerManager.isDeviceIdleMode)
            }
        }
        context.registerReceiver(receiver,
        IntentFilter(PowerManager.ACTION_DEVICE_IDLE_MODE_CHANGED))
    }
}
```

2.5 Quick Settings Tile Implementation

```kotlin
class HelixVpnTileService : TileService() {

    private val serviceConnection = object : ServiceConnection {
        override fun onServiceConnected(name: ComponentName?,
        service: IBinder?) {}
        override fun onServiceDisconnected(name: ComponentName?)
        {}
    }

    override fun onStartListening() {
        super.onStartListening()
        updateTileState()
```

```kotlin
}

override fun onClick() {
    // Collapse the Quick Settings panel
    if (Build.VERSION.SDK_INT >=
    Build.VERSION_CODES.UPSIDE_DOWN_CAKE) {
        // Android 14+ requires different approach
    }

    when (qsTile.state) {
        Tile.STATE_INACTIVE -> {
            // Connect to VPN
            val intent = Intent(this,
    HelixVpnService::class.java).apply {
                action = HelixVpnService.ACTION_CONNECT
                putExtra("config", getQuickConnectConfig())
            }
            startService(intent)
        }
        Tile.STATE_ACTIVE -> {
            // Disconnect VPN
            val intent = Intent(this,
    HelixVpnService::class.java).apply {
                action = HelixVpnService.ACTION_DISCONNECT
            }
            startService(intent)
        }
    }
}

private fun updateTileState() {
    val tile = qsTile ?: return

    when (HelixVpnService.connectionState) {
        VpnConnectionState.CONNECTED -> {
            tile.state = Tile.STATE_ACTIVE
            tile.label = "Helix VPN: ON"
            tile.icon = Icon.createWithResource(this,
    R.drawable.ic_tile_vpn_on)
        }
        VpnConnectionState.RECONNECTING -> {
            tile.state = Tile.STATE_ACTIVE
            tile.label = "Helix VPN: Reconnecting"
            tile.icon = Icon.createWithResource(this,
    R.drawable.ic_tile_vpn_connecting)
        }
        VpnConnectionState.KILL_SWITCH_ACTIVE -> {
            // Fail-closed: traffic is actively blocked, not
    just idle —
            // surface this distinctly so the user does not
    mistake a
            // blocked device for a working connection.
            tile.state = Tile.STATE_ACTIVE
```

```
                tile.label = "Helix VPN: Blocked (Kill Switch)"
                tile.icon = Icon.createWithResource(this,
        R.drawable.ic_tile_vpn_blocked)
            }
            VpnConnectionState.CONNECTING -> {
                tile.state = Tile.STATE_UNAVAILABLE
                tile.label = "Connecting..."
                tile.icon = Icon.createWithResource(this,
        R.drawable.ic_tile_vpn_connecting)
            }
            VpnConnectionState.DISCONNECTING -> {
                tile.state = Tile.STATE_UNAVAILABLE
                tile.label = "Disconnecting..."
                tile.icon = Icon.createWithResource(this,
        R.drawable.ic_tile_vpn_connecting)
            }
            VpnConnectionState.DISCONNECTED,
        VpnConnectionState.CONNECTION_FAILED -> {
                tile.state = Tile.STATE_INACTIVE
                tile.label = "Helix VPN"
                tile.icon = Icon.createWithResource(this,
        R.drawable.ic_tile_vpn_off)
            }
        }
        tile.updateTile()
    }

    private fun getQuickConnectConfig(): VpnConfig {
        // Load user's preferred quick-connect server
        val prefs = getSharedPreferences("helix_prefs",
        Context.MODE_PRIVATE)
        val serverId = prefs.getString("quick_connect_server",
        "auto") ?: "auto"
        return VpnConfig.quickConnect(serverId)
    }
}
```

AndroidManifest.xml registration:

```
<service
    android:name=".vpn.HelixVpnTileService"
    android:permission="android.permission.BIND_QUICK_SETTINGS_TILE"
    android:exported="true">
    <intent-filter>
        <action
        android:name="android.service.quicksettings.action.QS_TILE" /
        >
    </intent-filter>
    <meta-data
        android:name="android.service.quicksettings.ACTIVE_TILE"
        android:value="true" />
</service>
```

2.6 Kotlin Platform Channel Code

```kotlin
class MainActivity : FlutterActivity() {

    private val VPN_CHANNEL = "com.helix.vpn/native"
    private lateinit var vpnMethodChannel: MethodChannel

    override fun configureFlutterEngine(flutterEngine:
        FlutterEngine) {
        super.configureFlutterEngine(flutterEngine)

        vpnMethodChannel =
        MethodChannel(flutterEngine.dartExecutor.binaryMessenger,
        VPN_CHANNEL)
        vpnMethodChannel.setMethodCallHandler { call, result ->
            when (call.method) {
                "connect" -> handleConnect(call, result)
                "disconnect" -> handleDisconnect(result)
                "getStatus" -> handleGetStatus(result)
                "getStats" -> handleGetStats(result)
                "prepareVpn" -> handlePrepareVpn(result)
                "isVpnPrepared" -> handleIsVpnPrepared(result)
                "getInstalledApps" ->
        handleGetInstalledApps(result)
                "setSplitTunneling" ->
        handleSetSplitTunneling(call, result)
                "requestBatteryOptimizationExemption" ->
        handleBatteryExemption(result)
                else -> result.notImplemented()
            }
        }
    }

    private fun handleConnect(call: MethodCall, result:
        MethodChannel.Result) {
        val configMap = call.argument<Map<String, Any>>("config")
        val config = VpnConfig.fromMap(configMap ?: emptyMap())

        // Check VPN permission
        val intent = VpnService.prepare(this)
        if (intent != null) {
            // User hasn't granted VPN permission yet
            result.error("VPN_PERMISSION_REQUIRED", "User must
        grant VPN permission", null)
            return
        }

        // Start VPN service
        val serviceIntent = Intent(this,
        HelixVpnService::class.java).apply {
            action = HelixVpnService.ACTION_CONNECT
            putExtra("config", config)
        }
```

```kotlin
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            startForegroundService(serviceIntent)
        } else {
            startService(serviceIntent)
        }

        result.success(true)
    }

    private fun handleDisconnect(result: MethodChannel.Result) {
        val intent = Intent(this,
        HelixVpnService::class.java).apply {
            action = HelixVpnService.ACTION_DISCONNECT
        }
        startService(intent)
        result.success(true)
    }

    private fun handleGetStatus(result: MethodChannel.Result) {
        // String keys mirror the canonical `ConnectionStatus`
        values verbatim
        // (`MVP2_ARCHITECTURE.md` §5.6) so the Dart side's
        `ConnectionState`
        // enum (§5.3) can parse them without a platform-specific
        lookup table.
        val status = when (HelixVpnService.connectionState) {
            VpnConnectionState.CONNECTED -> "connected"
            VpnConnectionState.CONNECTING -> "connecting"
            VpnConnectionState.RECONNECTING -> "reconnecting"
            VpnConnectionState.KILL_SWITCH_ACTIVE ->
        "killSwitchActive"
            VpnConnectionState.DISCONNECTING -> "disconnecting"
            VpnConnectionState.DISCONNECTED -> "disconnected"
            VpnConnectionState.CONNECTION_FAILED ->
        "connectionFailed"
        }
        result.success(mapOf(
            "state" to status,
            "disconnectReason" to
        HelixVpnService.lastDisconnectReason
        ))
    }

    private fun handleGetStats(result: MethodChannel.Result) {
        val stats = rustBridge.getConnectionStats()
        result.success(mapOf(
            "uploadSpeed" to stats.uploadSpeed,
            "downloadSpeed" to stats.downloadSpeed,
            "totalUpload" to stats.totalUpload,
            "totalDownload" to stats.totalDownload,
            "duration" to stats.connectionDuration,
            "serverLocation" to stats.serverLocation,
```

```
        "protocol" to stats.protocol
    ))
}

private fun handlePrepareVpn(result: MethodChannel.Result) {
    val intent = VpnService.prepare(this)
    if (intent != null) {
        // Launch system VPN permission dialog
        vpnPermissionLauncher.launch(intent)
        result.success(false) // Permission not yet granted
    } else {
        result.success(true) // Already granted
    }
}

private fun handleIsVpnPrepared(result: MethodChannel.Result)
    {
    result.success(VpnService.prepare(this) == null)
}

private fun handleGetInstalledApps(result:
    MethodChannel.Result) {
    // Return list of installed apps for split tunneling
    configuration
    val pm = packageManager
    val apps =
    pm.getInstalledApplications(PackageManager.GET_META_DATA)
        .filter { app ->
            // Filter out system apps without launch intent
            app.flags and ApplicationInfo.FLAG_SYSTEM == 0 ||
            pm.getLaunchIntentForPackage(app.packageName) !=
    null
        }
        .map { app ->
            mapOf(
                "packageName" to app.packageName,
                "name" to
    pm.getApplicationLabel(app).toString(),
                "icon" to app.packageName // Flutter side
    loads icon via package_name
            )
        }
        .sortedBy { it["name"] as String }

    result.success(apps)
}

private fun handleSetSplitTunneling(call: MethodCall, result:
    MethodChannel.Result) {
    val mode = call.argument<String>("mode") ?: "off"
    val apps = call.argument<List<String>>("apps") ?:
    emptyList()
```

```kotlin
        // Save split tunneling preferences
        val prefs = getSharedPreferences("helix_prefs",
        Context.MODE_PRIVATE)
        prefs.edit()
            .putString("split_tunnel_mode", mode)
            .putStringSet("split_tunnel_apps", apps.toSet())
            .apply()

        result.success(true)
    }

    private fun handleBatteryExemption(result:
        MethodChannel.Result) {
        BatteryOptimizationHelper.requestBatteryOptimizationExemption(this)
        result.success(true)
    }

    private val vpnPermissionLauncher = registerForActivityResult(
        ActivityResultContracts.StartActivityForResult()
    ) { result ->
        if (result.resultCode == RESULT_OK) {

        vpnMethodChannel.invokeMethod("onVpnPermissionGranted",
        null)
        } else {

        vpnMethodChannel.invokeMethod("onVpnPermissionDenied",
        null)
        }
    }
}
```

2.7 Gradle Build Configuration

```groovy
// android/app/build.gradle
plugins {
    id "com.android.application"
    id "kotlin-android"
    id "dev.flutter.flutter-gradle-plugin"
    id "com.google.gms.google-services" // FCM (optional)
}

android {
    namespace "com.helix.vpn"
    compileSdk 35
    ndkVersion "25.2.9519653"

    compileOptions {
        sourceCompatibility JavaVersion.VERSION_17
        targetCompatibility JavaVersion.VERSION_17
    }
```

```
kotlinOptions {
    jvmTarget = '17'
}

defaultConfig {
    applicationId "com.helix.vpn"
    minSdk 26
    targetSdk 35
    versionCode 100
    versionName "1.0.0"

    multiDexEnabled true

    ndk {
        abiFilters 'arm64-v8a', 'armeabi-v7a', 'x86_64'
    }

    externalNativeBuild {
        cmake {
            arguments "-DANDROID_STL=c++_shared"
        }
    }
}

// Product flavors for different distribution channels
flavorDimensions += "distribution"

productFlavors {
    playStore {
        dimension "distribution"
        applicationIdSuffix ".play"
        buildConfigField "boolean",
    "ENABLE_DIRECT_BATTERY_REQUEST", "false"
    }
    fdroid {
        dimension "distribution"
        applicationIdSuffix ".fdroid"
        buildConfigField "boolean",
    "ENABLE_DIRECT_BATTERY_REQUEST", "true"
    }
    direct {
        dimension "distribution"
        buildConfigField "boolean",
    "ENABLE_DIRECT_BATTERY_REQUEST", "true"
    }
}

buildTypes {
    release {
        minifyEnabled true
        shrinkResources true
```

```
            proguardFiles getDefaultProguardFile('proguard-
        android-optimize.txt'), 'proguard-rules.pro'
            signingConfig signingConfigs.release
        }
        debug {
            minifyEnabled false
            signingConfig signingConfigs.debug
        }
    }

    externalNativeBuild {
        cmake {
            path "src/main/cpp/CMakeLists.txt"
            version "3.22.1"
        }
    }
}

flutter {
    source '../..'
}

dependencies {
    implementation "org.jetbrains.kotlin:kotlin-stdlib-jdk8"
    implementation 'androidx.core:core-ktx:1.13.0'
    implementation 'androidx.appcompat:appcompat:1.7.0'

    // Rust JNI bridge
    implementation project(':rust_bridge')

    // FCM for push notifications (optional)
    playStoreImplementation 'com.google.firebase:firebase-
        messaging:24.0.0'
}
```

2.8 ProGuard / R8 Rules

```
# android/app/proguard-rules.pro

# Keep VPN service and related classes
-keep class com.helix.vpn.vpn.** { *; }
-keep class com.helix.vpn.model.** { *; }

# Keep Rust JNI bridge classes
-keep class com.helix.vpn.rust.** { *; }
-keepclasseswithmembernames class * {
    native <methods>;
}

# Keep Flutter plugin classes
-keep class io.flutter.plugins.** { *; }
```

```
-keep class io.flutter.embedding.** { *; }

# Keep Parcelable implementations
-keepclassmembers class * implements android.os.Parcelable {
    public static final ** CREATOR;
}

# Keep serialized models for MethodChannel
-keepclassmembers class com.helix.vpn.model.VpnConfig { *; }
-keepclassmembers class com.helix.vpn.model.ConnectionStats { *; }

# Rust library
-keep class com.helix.vpn.rust.HelixRustBridge { *; }

# Don't warn about missing platform classes
-dontwarn java.awt.**
-dontwarn javax.swing.**
```

2.9 App Signing and Google Play Distribution

```
// android/app/build.gradle (signing configuration)
android {
    signingConfigs {
        release {
            storeFile
        file(System.getenv("HELIX_KEYSTORE_PATH") ?: "helix-
        release.keystore")
            storePassword System.getenv("HELIX_KEYSTORE_PASSWORD")
            keyAlias System.getenv("HELIX_KEY_ALIAS")
            keyPassword System.getenv("HELIX_KEY_PASSWORD")
        }
    }
}
```

AAB (Android App Bundle) for Google Play:

```
# Build release AAB
flutter build appbundle --release --flavor playStore

# Output: build/app/outputs/bundle/playStoreRelease/app-playStore-
        release.aab
```

APK for direct distribution / F-Droid:

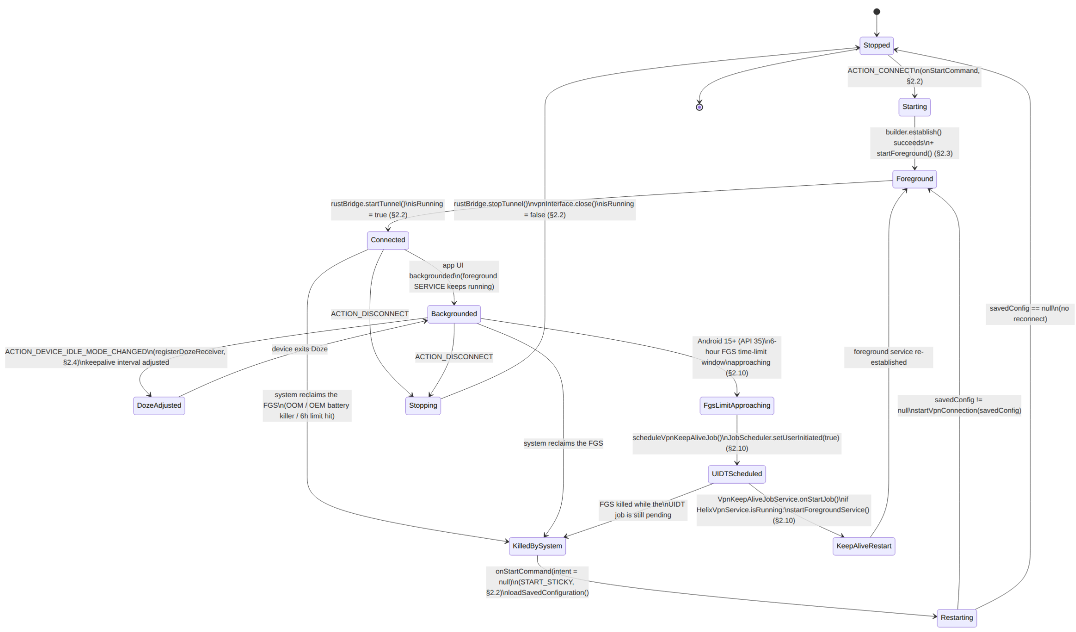
```
# Build split APKs per ABI
flutter build apk --release --split-per-abi --flavor fdroid

# Output:
# build/app/outputs/apk/fdroid/release/app-arm64-v8a-fdroid-
        release.apk
```

```
# build/app/outputs/apk/fdroid/release/app-armeabi-v7a-fdroid-
        release.apk
# build/app/outputs/apk/fdroid/release/app-x86_64-fdroid-
        release.apk
```

2.10 Android 15+ Foreground Service Restrictions

Starting with Android 15 (API 35), all foreground services share a **6-hour time limit** within a 24-hour window. For a VPN app, this requires adaptation:

The diagram below synthesizes the full Android foreground-service lifecycle described across this chapter — `onStartCommand`'s `ACTION_CONNECT` / `ACTION_DISCONNECT` / system-restart branches and `START_STICKY` return (§2.2), the persistent foreground notification (§2.3), the Doze-mode keepalive-interval adjustment (§2.4), and this section's Android 15+ UIDT job fallback for the 6-hour limit. It does not introduce any behavior not already described in prose/code above — it is a visual index into it.




```kotlin
object Android15ForegroundServiceHelper {

    /**
     * For Android 15+, use User-Initiated Data Transfer (UIDT)
        Jobs
     * as a fallback when foreground service time limit is
        reached.
     */
    @RequiresApi(Build.VERSION_CODES.UPSIDE_DOWN_CAKE)
    fun scheduleVpnKeepAliveJob(context: Context) {
        val jobScheduler =
        context.getSystemService(Context.JOB_SCHEDULER_SERVICE)
        as JobScheduler
```

```
        val jobInfo = JobInfo.Builder(
            JOB_ID_VPN_KEEPALIVE,
            ComponentName(context,
        VpnKeepAliveJobService::class.java)
        ).apply {
            setUserInitiated(true)
            setPersisted(true) // Survive reboots
            setRequiredNetworkType(JobInfo.NETWORK_TYPE_ANY)
            setMinimumLatency(TimeUnit.MINUTES.toMillis(5))
        }.build()

        jobScheduler.schedule(jobInfo)
    }

    /**
     * UIDT JobService that runs when the foreground service needs
        a break.
     * Maintains the VPN tunnel by keeping the Rust core active.
     */
    class VpnKeepAliveJobService : JobService() {
        override fun onStartJob(params: JobParameters?): Boolean {
            if (HelixVpnService.isRunning) {
                // Ensure VPN is still active; restart foreground
        service if needed
                val serviceIntent = Intent(this,
        HelixVpnService::class.java)
                startForegroundService(serviceIntent)
            }
            jobFinished(params, false)
            return true
        }

        override fun onStopJob(params: JobParameters?): Boolean {
            return true // Reschedule if stopped
        }
    }

    private const val JOB_ID_VPN_KEEPALIVE = 1001
}
```

---

3. iOS Client

3.1 Architecture

The iOS client uses a four-layer architecture with a separate Network Extension target:

```
┌─────────────────────────────────────────────────┐
│         Flutter UI Layer (Dart)                  │
│  - Main app bundle (com.helix.vpn)               │
```

```
│  - MethodChannel to native Swift code            │
├──────────────────────────────────────────────────┤
│     flutter_rust_bridge Layer                    │
│  - Dart FFI bindings (auto-generated)            │
├──────────────────────────────────────────────────┤
│     iOS App Host (Swift)                         │
│  - AppDelegate.swift                             │
│  - MethodChannel handler                         │
│  - NETunnelProviderManager configuration         │
│  - Keychain credential management                 │
│  - UniFFI Swift bindings to helix-core           │
├──────────────────────────────────────────────────┤
│  Network Extension (Separate Target)             │
│  Bundle: com.helix.vpn.packet-tunnel             │
│  - PacketTunnelProvider.swift (thin wrapper)     │
│  - NEPacketTunnelProvider subclass               │
│  - Rust core via C FFI (~15MB memory limit)      │
│  - App Groups for state sharing                  │
└──────────────────────────────────────────────────┘
```

3.2 NEPacketTunnelProvider Implementation

3.2.1 PacketTunnelProvider Extension

```swift
// PacketTunnelProvider/PacketTunnelProvider.swift
import NetworkExtension
import os.log

// Rust FFI imports (generated by UniFFI or hand-written)
import helix_core // Rust library C FFI bindings

class PacketTunnelProvider: NEPacketTunnelProvider {

    private let logger = OSLog(subsystem: "com.helix.vpn",
        category: "PacketTunnel")
    private var rustTunnelHandle:
        OpaquePointer? // Handle to Rust tunnel instance
    private var isTunnelActive = false

    // MARK: - Tunnel Lifecycle

    override func startTunnel(
        options: [String: NSObject]?,
        completionHandler: @escaping (Error?) -> Void
    ) {
        os_log("Starting tunnel...", log: logger, type: .info)

        // Extract configuration from protocolConfiguration
        guard let proto = protocolConfiguration as?
        NETunnelProviderProtocol,
```

```swift
          let providerConfig = proto.providerConfiguration
    else {
        completionHandler(NEVPNError(.configurationInvalid))
        return
    }

    // Retrieve configuration from App Group shared storage
    let config = loadConfiguration(from: providerConfig)

    // Configure tunnel network settings
    let settings = buildTunnelNetworkSettings(config: config)

    setTunnelNetworkSettings(settings) { [weak self] error in
        guard let self = self else { return }

        if let error = error {
            os_log("Failed to set tunnel settings: %{public}
    @",
                   log: self.logger, type: .error,
    error.localizedDescription)

    self.saveStateToAppGroup(state: .connectionFailed,
    config: nil)
            completionHandler(error)
            return
        }

        // Start the Rust tunnel core
        do {
            try self.startRustTunnel(config: config)
            self.isTunnelActive = true
            self.saveStateToAppGroup(state: .connected,
    config: config)
            completionHandler(nil)
        } catch {
            os_log("Failed to start Rust tunnel: %{public}@",
                   log: self.logger, type: .error,
    error.localizedDescription)
            // Initial-attempt failure maps to
    `connectionFailed`, never a
            // bespoke `error` state — see the `TunnelState`
    doc comment.

    self.saveStateToAppGroup(state: .connectionFailed,
    config: nil)
            completionHandler(error)
        }
    }
}

override func stopTunnel(
    with reason: NEProviderStopReason,
    completionHandler: @escaping () -> Void
```

```swift
) {
    os_log("Stopping tunnel, reason: %d", log: logger,
    type: .info, reason.rawValue)

    isTunnelActive = false
    stopRustTunnel()
    saveStateToAppGroup(state: .disconnected, config: nil)

    completionHandler()
}

// MARK: - App Extension IPC

override func handleAppMessage(
    _ messageData: Data,
    completionHandler: ((Data?) -> Void)?
) {
    // Handle IPC messages from the main app
    guard let message = try? JSONSerialization.jsonObject(
        with: messageData, options: []
    ) as? [String: Any] else {
        completionHandler?(nil)
        return
    }

    let action = message["action"] as? String ?? ""

    switch action {
    case "getStats":
        let stats = getRustTunnelStats()
        let responseData = try?
    JSONSerialization.data(withJSONObject: stats)
        completionHandler?(responseData)

    case "getStatus":
        let status: [String: Any] = [
            "state": isTunnelActive ? "connected" :
    "disconnected",
            "timestamp": Date().timeIntervalSince1970
        ]
        let responseData = try?
    JSONSerialization.data(withJSONObject: status)
        completionHandler?(responseData)

    default:
        completionHandler?(nil)
    }
}

// MARK: - Sleep / Wake Handling

override func sleep(completionHandler: @escaping () -> Void) {
```

```
        // Save any necessary state before device sleeps
        os_log("Device going to sleep", log: logger, type: .info)
        // Rust WireGuard core handles sleep silently (no
        keepalives)
        completionHandler()
    }

    override func wake() {
        // Restore state after device wakes
        os_log("Device waking up", log: logger, type: .info)
        if isTunnelActive {
            // Trigger a keepalive to verify tunnel is still alive
            rustTriggerKeepalive()
        }
    }

    // MARK: - Private Methods

    private func buildTunnelNetworkSettings(config: TunnelConfig)
        -> NEPacketTunnelNetworkSettings {
        let settings = NEPacketTunnelNetworkSettings(
            tunnelRemoteAddress: config.serverAddress
        )

        // IPv4 configuration
        let ipv4Settings = NEIPv4Settings(
            addresses: [config.tunnelIPv4Address],
            subnetMasks: [config.tunnelIPv4SubnetMask]
        )

        if config.routeAllTraffic {
            ipv4Settings.includedRoutes = [NEIPv4Route.default()]
        } else {
            ipv4Settings.includedRoutes =
        config.includedRoutes.map { route in
                NEIPv4Route(
                    destinationAddress: route.address,
                    subnetMask: route.subnetMask
                )
            }
        }

        // Exclude routes (split tunneling)
        ipv4Settings.excludedRoutes = config.excludedRoutes.map {
        route in
            NEIPv4Route(
                destinationAddress: route.address,
                subnetMask: route.subnetMask
            )
        }

        settings.ipv4Settings = ipv4Settings
```

```swift
    // IPv6 configuration (if enabled)
    if config.enableIPv6 {
        let ipv6Settings = NEIPv6Settings(
            addresses: [config.tunnelIPv6Address],
            networkPrefixLengths: [NSNumber(value:
config.tunnelIPv6PrefixLength)]
        )
        ipv6Settings.includedRoutes = [NEIPv6Route.default()]
        settings.ipv6Settings = ipv6Settings
    }

    // DNS configuration (critical for leak prevention)
    let dnsSettings = NEDNSSettings(servers:
config.dnsServers)
    dnsSettings.matchDomains = [""] // Use VPN DNS for all
queries
    settings.dnsSettings = dnsSettings

    // MTU
    settings.mtu = NSNumber(value: config.mtu)

    return settings
}

private func startRustTunnel(config: TunnelConfig) throws {
    // Convert config to JSON string for Rust core
    let configJson = try JSONEncoder().encode(config)
    guard let configString = String(data: configJson,
encoding: .utf8) else {
        throw TunnelError.invalidConfiguration
    }

    // Call into Rust core via FFI
    // The Rust core receives the TUN fd internally through
the NEPacketTunnelProvider
    let result = helix_core_start_tunnel(configString)

    if result < 0 {
        throw TunnelError.rustCoreError(code: result)
    }

    rustTunnelHandle = OpaquePointer(bitPattern: result)
}

private func stopRustTunnel() {
    if let handle = rustTunnelHandle {
        helix_core_stop_tunnel(handle)
        rustTunnelHandle = nil
    }
}

private func getRustTunnelStats() -> [String: Any] {
```

```
        guard let handle = rustTunnelHandle else {
            return [:]
        }

        let statsJson = helix_core_get_stats(handle)
        defer { free(statsJson) }

        guard let jsonData = String(cString:
        statsJson).data(using: .utf8),
              let stats = try? JSONSerialization.jsonObject(with:
        jsonData) as? [String: Any] else {
            return [:]
        }

        return stats
    }

    private func rustTriggerKeepalive() {
        guard let handle = rustTunnelHandle else { return }
        helix_core_trigger_keepalive(handle)
    }

    private func loadConfiguration(from providerConfig: [String:
        Any]) -> TunnelConfig {
        // Load configuration from providerConfiguration
        dictionary
        // and merge with App Group shared preferences
        var config = TunnelConfig()

        if let wgConfig = providerConfig["wgQuickConfig"] as?
        String {
            config.wireGuardConfig = wgConfig
        }
        if let serverAddress = providerConfig["serverAddress"]
        as? String {
            config.serverAddress = serverAddress
        }

        // Load from App Group UserDefaults
        if let sharedDefaults = UserDefaults(suiteName:
        "group.com.helix.vpn") {
            config.dnsServers =
        sharedDefaults.stringArray(forKey: "dns_servers")
                ?? ["1.1.1.1", "8.8.8.8"]
        }

        return config
    }

    private func saveStateToAppGroup(state: TunnelState, config:
        TunnelConfig?) {
        guard let sharedDefaults = UserDefaults(suiteName:
        "group.com.helix.vpn") else { return }
```

```swift
        sharedDefaults.set(state.rawValue, forKey: "tunnel_state")
        sharedDefaults.set(Date().timeIntervalSince1970, forKey:
        "last_state_update")

        if let config = config {
            sharedDefaults.set(config.serverAddress, forKey:
        "last_server")
        }

        sharedDefaults.synchronize()
    }

    /// Mirrors the canonical `ConnectionStatus` state machine
        owned by
    /// `helix-vpn-engine` (`MVP2_ARCHITECTURE.md` §5.6). The
        Network
    /// Extension does not invent its own states (no bespoke
        `error` value);
    /// a failed first-connection attempt is `connectionFailed`,
        and a
    /// post-connection failure is reported as `reconnecting` /
    /// `killSwitchActive` per the same state machine every other
        platform
    /// implements.
    enum TunnelState: String {
        case disconnected, connecting, connected, reconnecting
        case killSwitchActive, disconnecting, connectionFailed
    }

    enum TunnelError: Error {
        case invalidConfiguration
        case rustCoreError(code: Int32)
        case memoryLimitExceeded
    }
}
```

3.2.2 Memory Constraint Management

The iOS Network Extension has a strict ~15MB memory limit. Every byte counts:

```swift
// Memory management utilities for PacketTunnelProvider
import os.log

extension PacketTunnelProvider {

    /// Log current memory usage for debugging (remove in
        production)
    func logMemoryUsage() {
        var info = mach_task_basic_info()
```

```swift
        var count =
        mach_msg_type_number_t(MemoryLayout<mach_task_basic_info>.size)/
        4

        let kerr: kern_return_t = withUnsafeMutablePointer(to:
        &info) {
            $0.withMemoryRebound(to: integer_t.self, capacity: 1)
        {
                task_info(mach_task_self_,
        task_flavor_t(MACH_TASK_BASIC_INFO), $0, &count)
            }
        }

        if kerr == KERN_SUCCESS {
            let usedMB = Double(info.resident_size) / 1024.0 /
        1024.0
            os_log("Memory usage: %.2f MB", log: logger,
        type: .debug, usedMB)
        }
    }

    /// Optimization strategies for the 15MB limit:
    /// 1. Keep Swift wrapper minimal - all protocol logic in Rust
    /// 2. Use structs instead of classes where possible
    /// 3. Avoid closures that capture self strongly
    /// 4. Use autoreleasepool for tight loops
    /// 5. Disable logging in release builds
    /// 6. Use compressed data structures

    func optimizedPacketLoop() {
        autoreleasepool {
            // Process packets within autoreleasepool to prevent
            // accumulation of autoreleased objects
        }
    }
}
```

Memory Budget Allocation (15MB total): | Component | Budget | Notes | |-----------|--------|-------| | Rust WireGuard core | ~6-8 MB | GotaTun / BoringTun optimized build | | Swift runtime + NEPacketTunnelProvider | ~2-3 MB | Framework overhead | | Network buffers | ~2 MB | Packet I/O buffers | | Connection state | ~0.5 MB | Peer tracking, timers | | Logging (debug only) | ~0.5 MB | Strip in release | | Reserve | ~2-3 MB | Safety margin |

3.2.3 On-Demand Rule Configuration

```swift
import NetworkExtension

class OnDemandConfigurator {

    /// Configure VPN On-Demand rules for automatic connection
    static func setupOnDemandRules(
```

```swift
    manager: NETunnelProviderManager,
    config: OnDemandConfig
) {
    var rules: [NEOnDemandRule] = []

    // Rule 1: Connect on untrusted Wi-Fi networks
    if config.connectOnUntrustedWiFi {
        let wifiRule = NEOnDemandRuleConnect()
        wifiRule.interfaceTypeMatch = .wiFi
        wifiRule.ssidMatch = config.trustedWiFiNetworks
        wifiRule.action = .connect
        rules.append(wifiRule)
    }

    // Rule 2: Connect on cellular
    if config.connectOnCellular {
        let cellularRule = NEOnDemandRuleConnect()
        cellularRule.interfaceTypeMatch = .cellular
        cellularRule.action = .connect
        rules.append(cellularRule)
    }

    // Rule 3: Connect on any Ethernet (for iPad with dongle)
    if config.connectOnEthernet {
        let ethernetRule = NEOnDemandRuleConnect()
        ethernetRule.interfaceTypeMatch = .ethernet
        ethernetRule.action = .connect
        rules.append(ethernetRule)
    }

    // Rule 4: Disconnect on trusted Wi-Fi (e.g., home
network)
    if !config.trustedWiFiNetworks.isEmpty {
        let trustedWifiRule = NEOnDemandRuleDisconnect()
        trustedWifiRule.interfaceTypeMatch = .wiFi
        trustedWifiRule.ssidMatch = config.trustedWiFiNetworks
        trustedWifiRule.action = .disconnect
        rules.append(trustedWifiRule)
    }

    // Default rule: ignore (do nothing)
    let defaultRule = NEOnDemandRuleIgnore()
    rules.append(defaultRule)

    manager.onDemandRules = rules
    manager.isOnDemandEnabled = config.enabled
}

/// Domain-based on-demand for split tunneling evaluation
static func setupDomainEvaluationRules(
    manager: NETunnelProviderManager,
    domains: [String]
```

```swift
    ) {
        let evaluationRule = NEOnDemandRuleEvaluateConnection()
        evaluationRule.interfaceTypeMatch = .any

        let connectionRules = domains.map { domain in
            NEEvaluateConnectionRule(
                matchDomains: [domain],
                andAction: .connectIfNeeded
            )
        }

        evaluationRule.connectionRules = connectionRules
        manager.onDemandRules = [evaluationRule]
        manager.isOnDemandEnabled = true
    }
}

struct OnDemandConfig {
    var enabled: Bool = false
    var connectOnUntrustedWiFi: Bool = true
    var connectOnCellular: Bool = false
    var connectOnEthernet: Bool = true
    var trustedWiFiNetworks: [String] = []
}
```

3.3 App Groups for Container Sharing

```swift
// Shared container between main app and Network Extension
class AppGroupContainer {
    static let shared = AppGroupContainer()
    static let suiteName = "group.com.helix.vpn"

    private let userDefaults: UserDefaults?
    private let fileManager = FileManager.default

    private init() {
        userDefaults = UserDefaults(suiteName:
         AppGroupContainer.suiteName)
    }

    // MARK: - UserDefaults helpers

    func set(_ value: Any?, forKey key: String) {
        userDefaults?.set(value, forKey: key)
    }

    func string(forKey key: String) -> String? {
        return userDefaults?.string(forKey: key)
    }

    func bool(forKey key: String) -> Bool {
```

```swift
        return userDefaults?.bool(forKey: key) ?? false
    }

    // MARK: - File sharing

    var sharedContainerURL: URL? {
        return
         fileManager.containerURL(forSecurityApplicationGroupIdentifier:
            AppGroupContainer.suiteName)
    }

    func saveConfigurationFile(_ data: Data, filename: String)
         throws {
        guard let containerURL = sharedContainerURL else {
            throw AppGroupError.containerNotAvailable
        }

        let fileURL =
        containerURL.appendingPathComponent(filename)
        try data.write(to: fileURL, options: .atomic)
    }

    func loadConfigurationFile(filename: String) -> Data? {
        guard let containerURL = sharedContainerURL else { return
         nil }
        let fileURL =
        containerURL.appendingPathComponent(filename)
        return try? Data(contentsOf: fileURL)
    }

    enum AppGroupError: Error {
        case containerNotAvailable
        case writeFailed
        case readFailed
    }
}
```

3.4 Network Extension Entitlement

```xml
<!-- Runner/Runner.entitlements (Main App) -->
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <!-- App Groups for main app and extension sharing -->
    <key>com.apple.security.application-groups</key>
    <array>
        <string>group.com.helix.vpn</string>
    </array>
```

```xml
        <!-- Network Extension entitlement (self-serve since Nov
        2016) -->
    <key>com.apple.developer.networking.networkextension</key>
    <array>
        <string>packet-tunnel-provider</string>
    </array>

    <!-- Keychain sharing -->
    <key>keychain-access-groups</key>
    <array>
        <string>$(AppIdentifierPrefix)com.helix.vpn</string>
    </array>
</dict>
</plist>

<!-- PacketTunnelProvider/PacketTunnelProvider.entitlements
        (Extension) -->
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>com.apple.security.application-groups</key>
    <array>
        <string>group.com.helix.vpn</string>
    </array>
</dict>
</plist>
```

3.5 Keychain Integration for Credentials

```swift
import Security
import LocalAuthentication

class KeychainCredentialStore {

    static let shared = KeychainCredentialStore()
    private let service = "com.helix.vpn.credentials"

    /// Save VPN credentials with optional biometric protection
    func saveCredentials(
        account: String,
        password: String,
        biometricRequired: Bool = false
    ) -> OSStatus {
        // Delete existing item first
        deleteCredentials(account: account)

        var query: [String: Any] = [
            kSecClass as String: kSecClassGenericPassword,
            kSecAttrAccount as String: account,
```

```swift
        kSecAttrService as String: service,
        kSecValueData as String: password.data(using: .utf8)!,
        kSecAttrAccessible as String:
    kSecAttrAccessibleWhenUnlockedThisDeviceOnly
    ]

    // Add biometric protection if requested
    if biometricRequired {
        var error: Unmanaged<CFError>?
        guard let accessControl =
    SecAccessControlCreateWithFlags(
            kCFAllocatorDefault,
            kSecAttrAccessibleWhenUnlockedThisDeviceOnly,
            .biometryCurrentSet,
            &error
        ) else {
            return errSecParam
        }
        query[kSecAttrAccessControl as String] = accessControl
        // Remove accessible attribute when using access
    control
        query.removeValue(forKey: kSecAttrAccessible as
    String)
    }

    return SecItemAdd(query as CFDictionary, nil)
}

/// Retrieve credentials (triggers biometric prompt if
    protected)
func retrieveCredentials(account: String) -> String? {
    let query: [String: Any] = [
        kSecClass as String: kSecClassGenericPassword,
        kSecAttrAccount as String: account,
        kSecAttrService as String: service,
        kSecReturnData as String: true,
        kSecMatchLimit as String: kSecMatchLimitOne,
        kSecUseOperationPrompt as String: "Authenticate to
    access VPN credentials"
    ]

    var result: CFTypeRef?
    let status = SecItemCopyMatching(query as CFDictionary,
    &result)

    guard status == errSecSuccess,
          let data = result as? Data,
          let password = String(data: data, encoding: .utf8)
    else {
        return nil
    }

    return password
```

```swift
    }

    /// Delete stored credentials
    func deleteCredentials(account: String) -> OSStatus {
        let query: [String: Any] = [
            kSecClass as String: kSecClassGenericPassword,
            kSecAttrAccount as String: account,
            kSecAttrService as String: service
        ]
        return SecItemDelete(query as CFDictionary)
    }

    /// Save WireGuard private key securely
    func saveWireGuardPrivateKey(_ privateKey: String) ->
        OSStatus {
        return saveCredentials(account: "wg_private_key",
        password: privateKey)
    }

    /// Retrieve WireGuard private key
    func getWireGuardPrivateKey() -> String? {
        return retrieveCredentials(account: "wg_private_key")
    }
}
```

3.6 Control Center Integration

iOS VPN apps automatically appear in Settings > VPN and can be toggled via Control Center. The app provides the configuration; the system handles the UI:

```swift
import NetworkExtension

class VpnConfigurationManager {

    static let shared = VpnConfigurationManager()

    /// Load or create the VPN configuration
    func loadOrCreateConfiguration(
        completion: @escaping (NETunnelProviderManager?, Error?)
        -> Void
    ) {
        NETunnelProviderManager.loadAllFromPreferences {
        managers, error in
            if let error = error {
                completion(nil, error)
                return
            }

            // Return existing configuration if found
            if let existingManager = managers?.first(where: {
        manager in
```

```swift
            (manager.protocolConfiguration as?
    NETunnelProviderProtocol)?
                .providerBundleIdentifier ==
    "com.helix.vpn.packet-tunnel"
        }) {
            completion(existingManager, nil)
            return
        }

        // Create new configuration
        let newManager = self.createNewConfiguration()
        newManager.saveToPreferences { error in
            if let error = error {
                completion(nil, error)
                return
            }
            // Must reload after saving
            NETunnelProviderManager.loadAllFromPreferences {
    _, _ in
                completion(newManager, nil)
            }
        }
    }
}

private func createNewConfiguration() ->
    NETunnelProviderManager {
    let manager = NETunnelProviderManager()

    let protocolConfig = NETunnelProviderProtocol()
    protocolConfig.providerBundleIdentifier =
    "com.helix.vpn.packet-tunnel"
    protocolConfig.serverAddress = "helix-vpn.example.com"
    protocolConfig.providerConfiguration = [
        "wgQuickConfig": "", // Populated at connection time
        "protocol": "wireguard"
    ]

    manager.protocolConfiguration = protocolConfig
    manager.localizedDescription = "Helix VPN"
    manager.isEnabled = true

    return manager
}

/// Start the VPN tunnel
func connect(
    manager: NETunnelProviderManager,
    config: TunnelConfig,
    completion: @escaping (Error?) -> Void
) {
    // Update protocol configuration with current server
```

```
        if let proto = manager.protocolConfiguration as?
        NETunnelProviderProtocol {
            var providerConfig = proto.providerConfiguration ??
        [:]
            providerConfig["serverAddress"] =
        config.serverAddress as NSString
            providerConfig["wgQuickConfig"] =
        config.wireGuardConfig as NSString
            proto.providerConfiguration = providerConfig
            proto.serverAddress = config.serverAddress
        }

        manager.saveToPreferences { error in
            if let error = error {
                completion(error)
                return
            }

            do {
                try manager.connection.startVPNTunnel()
                completion(nil)
            } catch {
                completion(error)
            }
        }
    }

    /// Stop the VPN tunnel
    func disconnect(manager: NETunnelProviderManager) {
        manager.connection.stopVPNTunnel()
    }

    /// Monitor connection status
    var connectionStatus: NEVPNStatus {
        //
        Returns: .invalid, .disconnected, .connecting, .connected, .reasserting, .
        return NETunnelProviderManager().connection.status
    }
}
```

3.7 iOS-Specific UI Adaptations

```
// lib/ui/platform/ios_ui_adapters.dart
import 'package:flutter/cupertino.dart';
import 'package:flutter/material.dart';
import 'dart:io' show Platform;

/// iOS-specific UI adaptations to match platform conventions
class IOSUiAdapter {

  /// Use Cupertino-style navigation bar on iOS
  static PreferredSizeWidget buildAppBar({
```

```
  required BuildContext context,
  required String title,
  List<Widget>? actions,
  Widget? leading,
  bool centerTitle = true,
}) {
  if (Platform.isIOS) {
    return CupertinoNavigationBar(
      middle: Text(title),
      trailing: actions != null && actions.isNotEmpty
          ? Row(mainAxisSize: MainAxisSize.min, children:
      actions)
          : null,
      leading: leading,
      backgroundColor:
      CupertinoColors.systemBackground.withOpacity(0.9),
    ) as PreferredSizeWidget;
  }

  return AppBar(
    title: Text(title),
    centerTitle: centerTitle,
    actions: actions,
    leading: leading,
  );
}

/// Platform-appropriate button style
static Widget buildPrimaryButton({
  required String label,
  required VoidCallback onPressed,
  bool isDestructive = false,
}) {
  if (Platform.isIOS) {
    return CupertinoButton.filled(
      onPressed: onPressed,
      child: Text(label),
    );
  }

  return ElevatedButton(
    onPressed: onPressed,
    style: isDestructive
        ? ElevatedButton.styleFrom(backgroundColor: Colors.red)
        : null,
    child: Text(label),
  );
}

/// Platform-appropriate settings list
static Widget buildSettingsList({
  required List<SettingsItem> items,
```

```dart
}) {
  if (Platform.isIOS) {
    return CupertinoFormSection.insetGrouped(
      children: items.map((item) => CupertinoFormRow(
        prefix: Row(
          children: [
            if (item.icon != null)
              Padding(
                padding: const EdgeInsets.only(right: 12),
                child: Icon(item.icon, color:
      CupertinoColors.systemBlue),
              ),
            Text(item.title),
          ],
        ),
        child: item.trailing ?? const SizedBox.shrink(),
        helper: item.subtitle != null
            ? Text(item.subtitle!, style: const
      TextStyle(fontSize: 12))
            : null,
      )).toList(),
    );
  }

  return ListView.builder(
    itemCount: items.length,
    itemBuilder: (context, index) {
      final item = items[index];
      return ListTile(
        leading: item.icon != null ? Icon(item.icon) : null,
        title: Text(item.title),
        subtitle: item.subtitle != null ?
      Text(item.subtitle!) : null,
        trailing: item.trailing ?? const
      Icon(Icons.chevron_right),
        onTap: item.onTap,
      );
    },
  );
}

/// Platform-appropriate connection toggle (large central
      button)
static Widget buildConnectionToggle({
  required bool isConnected,
  required VoidCallback onToggle,
  required bool isConnecting,
}) {
  if (Platform.isIOS) {
    return CupertinoButton(
      onPressed: isConnecting ? null : onToggle,
      child: Container(
        width: 120,
```

```
        height: 120,
        decoration: BoxDecoration(
          shape: BoxShape.circle,
          color: isConnected
              ? CupertinoColors.activeGreen
              : CupertinoColors.systemGrey,
          boxShadow: [
            BoxShadow(
              color: (isConnected
                  ? CupertinoColors.activeGreen
                  :
      CupertinoColors.systemGrey).withOpacity(0.3),
              blurRadius: 20,
              spreadRadius: 5,
            ),
          ],
        ),
        child: Center(
          child: isConnecting
              ? const CupertinoActivityIndicator(color:
      CupertinoColors.white)
              : Icon(
                  isConnected ? CupertinoIcons.check_mark :
      CupertinoIcons.power,
                  color: CupertinoColors.white,
                  size: 48,
                ),
        ),
      ),
    );
  }

  // Material design fallback
  return FloatingActionButton.large(
    onPressed: isConnecting ? null : onToggle,
    backgroundColor: isConnected ? Colors.green : Colors.grey,
    child: isConnecting
        ? const CircularProgressIndicator(color: Colors.white)
        : Icon(isConnected ? Icons.check :
      Icons.power_settings_new, size: 48),
  );
 }
}

class SettingsItem {
  final String title;
  final String? subtitle;
  final IconData? icon;
  final Widget? trailing;
  final VoidCallback? onTap;

  SettingsItem({
```

```
    required this.title,
    this.subtitle,
    this.icon,
    this.trailing,
    this.onTap,
  });
}
```

3.8 App Store Submission Requirements

3.8.1 VPN App Review Guidelines

Apple scrutinizes VPN apps heavily. Requirements include:

- **Privacy Policy**: Must accurately describe all data collection, use, and retention practices. Must be accessible from both the App Store listing and within the app.
- **Network Extension Justification**: The app must demonstrate a legitimate use case for the Network Extension entitlement.
- **User Authorization**: User must explicitly approve the VPN configuration on first use (system-enforced).
- **Battery Impact**: Apps that drain battery excessively may be rejected. WireGuard typically passes due to its efficient design.
- **No Misleading Claims**: Avoid terms like "military-grade encryption" without substantiation.
- **App Tracking Transparency (ATT)**: If collecting any tracking data, ATT framework must be implemented.

3.8.2 Required Entitlements and Provisioning

Requirement	Details
Developer Account	Apple Developer Program ($99/year)
Entitlement	`com.apple.developer.networking.networkextension` with `packet-tunnel-provider`
App Groups	`group.com.helix.vpn` for app/extension data sharing
Provisioning Profile	Must include Network Extension entitlement
Signing	Distribution certificate for App Store

3.8.3 Info.plist Requirements

```
<!-- iOS Runner/Info.plist additions -->
<key>NSLocalNetworkUsageDescription</key>
<string>Helix VPN needs to manage your network connection to
        establish a secure VPN tunnel.</string>
<key>UIBackgroundModes</key>
```

```xml
<array>
    <string>network-authentication</string>
</array>
<key>NSFaceIDUsageDescription</key>
<string>Helix VPN uses Face ID to protect your VPN credentials.</
        string>
```

3.9 TestFlight Distribution

```bash
# Build iOS release archive
flutter build ipa --release --export-options-plist=ios/
        exportOptions.plist

# Upload to App Store Connect / TestFlight
xcrun altool --upload-app \
  --type ios \
  --file build/ios/ipa/HelixVPN.ipa \
  --apiKey "YOUR_API_KEY" \
  --apiIssuer "YOUR_ISSUER_ID"

# exportOptions.plist
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>method</key>
    <string>app-store</string>
    <key>teamID</key>
    <string>YOUR_TEAM_ID</string>
    <key>uploadSymbols</key>
    <true/>
    <key>uploadBitcode</key>
    <false/>
</dict>
</plist>
```

---

4. HarmonyOS Client

4.1 Architecture

The HarmonyOS client follows a similar pattern to Android but uses HarmonyOS-specific APIs:

```
┌─────────────────────────────────────────────┐
│          Flutter UI Layer (Dart)             │
│  - Flutter 3.22.0-ohos embedding layer       │
├─────────────────────────────────────────────┤
│      flutter_rust_bridge Layer               │
```

```
│  - Dart FFI bindings to Rust core              │
├──────────────────────────────────────────────┤
│     HarmonyOS Native Layer (ArkTS)             │
│  - VpnExtensionAbility                         │
│  - N-API bridge to Rust                        │
│  - Form card (service widget)                  │
├──────────────────────────────────────────────┤
│         Shared Rust Core                       │
│  - helix-core via N-API                        │
│  - WireGuard (primary) / Shadowsocks /          │
│    MASQUE / Multi-Hop (OpenVPN reserved,        │
│    unimplemented — see §1.1)                   │
└──────────────────────────────────────────────┘
```

4.2 HarmonyOS VPN API (VpnExtensionAbility)

```
// entry/src/main/ets/vpnability/HelixVpnAbility.ets
import { VpnExtensionAbility } from '@kit.NetworkKit';
import { Want } from '@kit.AbilityKit';
import vpn from '@ohos.net.vpn';
import hilog from '@ohos.hilog';

// N-API imports for Rust core
import { HelixRustBridge } from '../rust/HelixRustBridge';

const TAG = 'HelixVpnAbility';

export default class HelixVpnAbility extends VpnExtensionAbility {
  private vpnConnection: vpn.VpnConnection | null = null;
  private rustBridge: HelixRustBridge | null = null;
  private tunFd: number = -1;

  onCreate(want: Want): void {
    hilog.info(0x0000, TAG, 'HelixVpnAbility onCreate');

    // Initialize Rust bridge via N-API
    this.rustBridge = new HelixRustBridge();

    // Extract configuration from want parameters
    const config = want.parameters as VpnConfig;
    if (config) {
      this.setupVpn(config);
    }
  }

  onDestroy(): void {
    hilog.info(0x0000, TAG, 'HelixVpnAbility onDestroy');
    this.teardownVpn();
  }

  private async setupVpn(config: VpnConfig): Promise<void> {
```

```
  try {
    // Create VPN connection through system API
    this.vpnConnection = vpn.createVpnConnection(this.context);

    // Build VPN configuration
    const vpnConfig: vpn.VpnConfig = {
      addresses: [{
        address: { address: config.tunnelAddress, family:
      1 }, // AF_INET
        prefixLength: config.tunnelPrefixLength
      }],
      mtu: config.mtu || 1400,
      dnsAddresses: config.dnsServers,
      routes: config.routes.map(route => ({
        address: { address: route.address, family:
      route.family },
        prefixLength: route.prefixLength
      })),
      isBlocking: config.killSwitchEnabled || false, // Kill
      switch
      isIPv4Accepted: true,
      isIPv6Accepted: config.enableIPv6 || false,
      trustedApplications: config.allowedApplications || [],
      blockedApplications: config.blockedApplications || [],
    };

    // Establish VPN tunnel (returns TUN file descriptor)
    this.tunFd = await this.vpnConnection.setUp(vpnConfig);
    hilog.info(0x0000, TAG, `VPN setUp success, tunFd: $
      {this.tunFd}`);

    // Pass TUN fd to Rust core for packet processing
    if (this.rustBridge) {
      this.rustBridge.startTunnel(this.tunFd,
      JSON.stringify(config));
    }

    // Save connection state
    this.saveConnectionState('connected', config);

  } catch (error) {
    hilog.error(0x0000, TAG, `VPN setup failed: $
      {JSON.stringify(error)}`);
    // Initial-attempt failure maps to the canonical
      `connectionFailed`
    // state (`MVP2_ARCHITECTURE.md` §5.6), never a HarmonyOS-
      only 'error'
    // value — see the `TunnelState` type below.
    this.saveConnectionState('connectionFailed', config);
  }
}

private async teardownVpn(): Promise<void> {
```

```
    // Stop Rust core
    if (this.rustBridge) {
      this.rustBridge.stopTunnel();
    }

    // Destroy VPN connection
    if (this.vpnConnection) {
      try {
        await this.vpnConnection.destroy();
        hilog.info(0x0000, TAG, 'VPN connection destroyed');
      } catch (error) {
        hilog.error(0x0000, TAG, `VPN destroy error: ${JSON.stringify(error)}`);
      }
      this.vpnConnection = null;
    }

    this.tunFd = -1;
    this.saveConnectionState('disconnected', null);
  }

  private saveConnectionState(state: string, config: VpnConfig | null): void {
    // Use AppStorage or preferences for state persistence
    // so the main app can read the current VPN status
  }
}

// VpnConfig interface
interface VpnConfig {
  tunnelAddress: string;
  tunnelPrefixLength: number;
  mtu?: number;
  dnsServers: string[];
  routes: RouteConfig[];
  killSwitchEnabled?: boolean;
  enableIPv6?: boolean;
  allowedApplications?: string[];
  blockedApplications?: string[];
}

interface RouteConfig {
  address: string;
  family: number;
  prefixLength: number;
}
```

4.3 N-API Bridge for Rust

```
// entry/src/main/ets/rust/HelixRustBridge.ets
// N-API bridge module declaration
```

```
import napi from 'libnapi.so'; // Native library

const TAG = 'HelixRustBridge';

/**
 * Bridge to helix-core Rust library via N-API.
 * This is the HarmonyOS equivalent of JNI (Android) or C FFI
       (iOS).
 */
export class HelixRustBridge {
  private nativeHandle: napi.NativeHandle;

  constructor() {
    // Initialize the native Rust module
    this.nativeHandle = napi.initHelixCore();
  }

  /**
   * Start the VPN tunnel with the given TUN fd and configuration.
   * @param tunFd - File descriptor of the TUN device
   * @param configJson - JSON string containing tunnel
       configuration
   */
  startTunnel(tunFd: number, configJson: string): number {
    return napi.helixStartTunnel(this.nativeHandle, tunFd,
       configJson);
  }

  /**
   * Stop the active VPN tunnel.
   */
  stopTunnel(): number {
    return napi.helixStopTunnel(this.nativeHandle);
  }

  /**
   * Get current connection statistics.
   */
  getStats(): ConnectionStats {
    const statsJson = napi.helixGetStats(this.nativeHandle);
    return JSON.parse(statsJson) as ConnectionStats;
  }

  /**
   * Get current tunnel state.
   */
  getState(): TunnelState {
    return napi.helixGetState(this.nativeHandle) as TunnelState;
  }

  /**
   * Release native resources.
```

```
   */
  destroy(): void {
    napi.helixDestroy(this.nativeHandle);
  }
}

interface ConnectionStats {
  uploadSpeed: number;
  downloadSpeed: number;
  totalUpload: number;
  totalDownload: number;
  duration: number;
}

// Mirrors the canonical `ConnectionStatus` state machine owned by
// `helix-vpn-engine` (`MVP2_ARCHITECTURE.md` §5.6) — no ability-
        invented
// states (no bespoke 'error' value; see `setupVpn`'s catch block
        above).
type TunnelState = 'disconnected' | 'connecting' | 'connected' |
        'reconnecting'
  | 'killSwitchActive' | 'disconnecting' | 'connectionFailed';
```

Native C++ N-API binding layer:

```
// entry/src/main/cpp/napi_helix_bridge.cpp
#include <napi/native_api.h>
#include <hilog/log.h>
#include <string>
#include "helix_core.h" // Rust-generated C header

#define LOG_TAG "HelixNAPI"

// Forward declarations of Rust FFI functions
extern "C" {
    void* helix_core_init();
    int32_t helix_start_tunnel(void* handle, int32_t tun_fd,
        const char* config_json);
    int32_t helix_stop_tunnel(void* handle);
    const char* helix_get_stats(void* handle);
    int32_t helix_get_state(void* handle);
    void helix_destroy(void* handle);
    void helix_free_string(const char* str);
}

static napi_value NAPI_InitHelixCore(napi_env env,
        napi_callback_info info) {
    void* handle = helix_core_init();
    napi_value result;
    napi_create_bigint_uint64(env,
        reinterpret_cast<uint64_t>(handle), &result);
    return result;
}
```

```cpp
static napi_value NAPI_HelixStartTunnel(napi_env env,
        napi_callback_info info) {
    size_t argc = 3;
    napi_value args[3] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);

    // Extract handle
    uint64_t handle_val;
    napi_get_value_bigint_uint64(env, args[0], &handle_val,
        nullptr);
    void* handle = reinterpret_cast<void*>(handle_val);

    // Extract TUN fd
    int32_t tun_fd;
    napi_get_value_int32(env, args[1], &tun_fd);

    // Extract config JSON
    size_t config_len;
    napi_get_value_string_utf8(env, args[2], nullptr, 0,
        &config_len);
    std::string config_json(config_len, '\0');
    napi_get_value_string_utf8(env, args[2], &config_json[0],
        config_len + 1, &config_len);

    int32_t result = helix_start_tunnel(handle, tun_fd,
        config_json.c_str());

    napi_value napi_result;
    napi_create_int32(env, result, &napi_result);
    return napi_result;
}

static napi_value NAPI_HelixGetStats(napi_env env,
        napi_callback_info info) {
    size_t argc = 1;
    napi_value args[1] = {nullptr};
    napi_get_cb_info(env, info, &argc, args, nullptr, nullptr);

    uint64_t handle_val;
    napi_get_value_bigint_uint64(env, args[0], &handle_val,
        nullptr);
    void* handle = reinterpret_cast<void*>(handle_val);

    const char* stats = helix_get_stats(handle);
    napi_value result;
    napi_create_string_utf8(env, stats, NAPI_AUTO_LENGTH,
        &result);
    helix_free_string(stats);

    return result;
}
```

```cpp
// Module registration
static napi_value Init(napi_env env, napi_value exports) {
    napi_property_descriptor desc[] = {
        {"initHelixCore", nullptr, NAPI_InitHelixCore, nullptr,
         nullptr, nullptr, napi_default, nullptr},
        {"helixStartTunnel", nullptr, NAPI_HelixStartTunnel,
         nullptr, nullptr, nullptr, napi_default, nullptr},
        {"helixGetStats", nullptr, NAPI_HelixGetStats, nullptr,
         nullptr, nullptr, napi_default, nullptr},
        // ... other methods
    };
    napi_define_properties(env, exports, sizeof(desc) /
         sizeof(desc[0]), desc);
    return exports;
}

static napi_module helixModule = {
    .nm_version = 1,
    .nm_flags = 0,
    .nm_filename = nullptr,
    .nm_register_func = Init,
    .nm_modname = "libnapi",
    .nm_priv = nullptr,
    .reserved = {0}
};

extern "C" __attribute__((constructor)) void RegisterModule() {
    napi_module_register(&helixModule);
}
```

4.4 DevEco Studio Build Setup

```json5
// build-profile.json5
{
  "app": {
    "signingConfigs": [],
    "compileSdkVersion": 12,
    "compatibleSdkVersion": 12,
    "products": [
      {
        "name": "default",
        "signingConfig": "default",
        "compatibleSdkVersion": "12",
        "runtimeOS": "HarmonyOS"
      }
    ],
    "buildOption": {
      "strictMode": {
        "caseSensitiveCheck": true,
        "useNormalizedOHMUrl": true
      }
    }
```

```
  },
  "modules": [
    {
      "name": "entry",
      "srcPath": "./entry",
      "targets": [
        {
          "name": "default",
          "applyToProducts": ["default"]
        }
      ]
    }
  ]
}

// entry/build-profile.json5
{
  "apiType": "stageMode",
  "buildOption": {
    "externalOptions": {
      "ignore ossAudit": true
    },
    "arkOptions": {
      "buildProfileFields": {
        "HARMONY_OS_BUILD": "true"
      }
    },
    "nativeOptions": {
      "libraries": [
        {
          "name": "libnapi.so",
          "path": "./src/main/cpp/",
          "napi": true
        }
      ]
    }
  },
  "buildOptionSet": [
    {
      "name": "release",
      "arkOptions": {
        "obfuscation": {
          "ruleOptions": {
            "enable": true,
            "files": ["./obfuscation-rules.txt"]
          }
        }
      }
    }
  ],
  "targets": [
    {
```

```
      "name": "default"
    }
  ]
}
```

```
// entry/module.json5
{
  "module": {
    "name": "entry",
    "type": "entry",
    "description": "$string:module_desc",
    "mainElement": "EntryAbility",
    "deviceTypes": ["phone", "tablet"],
    "deliveryWithInstall": true,
    "installationFree": false,
    "pages": "$profile:main_pages",
    "abilities": [
      {
        "name": "EntryAbility",
        "srcEntry": "./ets/entryability/EntryAbility.ets",
        "description": "$string:EntryAbility_desc",
        "icon": "$media:icon",
        "label": "$string:EntryAbility_label",
        "startWindowIcon": "$media:startIcon",
        "startWindowBackground": "$color:start_window_background",
        "exported": true,
        "skills": [
          {
            "entities": ["entity.system.home"],
            "actions": ["action.system.home"]
          }
        ]
      }
    ],
    "extensionAbilities": [
      {
        "name": "HelixVpnAbility",
        "srcEntry": "./ets/vpnability/HelixVpnAbility.ets",
        "type": "vpn",
        "exported": false
      }
    ],
    "requestPermissions": [
      {
        "name": "ohos.permission.MANAGE_VPN",
        "reason": "$string:permission_vpn_reason",
        "usedScene": {
          "abilities": ["HelixVpnAbility"],
          "when": "inuse"
        }
      },
      {
```

```
        "name": "ohos.permission.INTERNET"
      },
      {
        "name": "ohos.permission.GET_NETWORK_INFO"
      }
    ]
  }
}
```

4.5 ohos-specific Flutter Plugins

```yaml
# pubspec.yaml - HarmonyOS-specific dependencies
name: helix_vpn
version: 1.0.0
environment:
  sdk: '>=3.0.0 <4.0.0'
  flutter: ">=3.22.0"

dependencies:
  flutter:
    sdk: flutter

  # Core dependencies (all platforms)
  flutter_rust_bridge: ^2.0.0
  riverpod: ^2.5.0
  dio: ^5.4.0
  shared_preferences: ^2.2.0

  # Conditional HarmonyOS dependencies
  connectivity_plus: ^5.0.0

dependency_overrides:
  # Use HarmonyOS-compatible versions
  path_provider:
    git:
      url: https://gitee.com/openharmony-sig/flutter_packages.git
      path: packages/path_provider/path_provider

dev_dependencies:
  flutter_test:
    sdk: flutter
  build_runner: ^2.4.0
  freezed: ^2.4.0

flutter:
  uses-material-design: true
  assets:
    - assets/images/
    - assets/flags/
```

4.6 AppGallery Distribution

```bash
# Build HarmonyOS HAP package
flutter build hap --release

# Output: build/ohos/outputs/default/entry-default-signed.hap

# AppGallery Connect upload
# 1. Register app at https://developer.huawei.com/consumer/en/
        appgallery/
# 2. Sign the HAP with Huawei certificates
# 3. Upload through AppGallery Connect web interface or CLI

# Signing configuration requires:
# - Huawei Developer account
# - App signing certificate (.cer)
# - App signing private key (.p12)
# - Profile file (.p7b)
```

4.7 Form Card (Service Widget) for Quick Connect

```typescript
// entry/src/main/ets/widget/pages/HelixWidgetCard.ets
import { AppRouter } from '@kit.ArkUI';
import formBindingData from '@ohos.app.form.formBindingData';
import FormExtensionAbility from
        '@ohos.app.form.FormExtensionAbility';
import formProvider from '@ohos.app.form.formProvider';

// Widget form data
interface WidgetFormData {
  vpnStatus: string;
  serverLocation: string;
  connectButtonText: string;
  statusColor: string;
}

export default class HelixWidgetAbility extends
        FormExtensionAbility {

  onCreate(want: Want): formBindingData.FormBindingData {
    // Initial widget data
    const formData: WidgetFormData = {
      vpnStatus: 'Disconnected',
      serverLocation: 'Tap to connect',
      connectButtonText: 'Connect',
      statusColor: '#808080'
    };

    return formBindingData.createFormBindingData(formData);
  }
```

```
  onUpdate(formId: string): void {
    // Read current VPN state from preferences
    const vpnState = this.readVpnState();

    const formData: WidgetFormData = {
      vpnStatus: vpnState.connected ? 'Connected' :
        'Disconnected',
      serverLocation: vpnState.serverLocation || 'Tap to connect',
      connectButtonText: vpnState.connected ? 'Disconnect' :
        'Connect',
      statusColor: vpnState.connected ? '#00C853' : '#808080'
    };

    formProvider.updateForm(formId,
        formBindingData.createFormBindingData(formData));
  }

  onFormEvent(formId: string, message: string): void {
    const event = JSON.parse(message);

    if (event.action === 'toggle') {
      // Launch VPN ability to toggle connection
      const want: Want = {
        bundleName: 'com.helix.vpn',
        abilityName: 'HelixVpnAbility',
        parameters: {
          action: event.connected ? 'disconnect' : 'connect'
        }
      };
      this.context.startAbility(want);
    }
  }

  private readVpnState(): { connected: boolean; serverLocation?:
        string } {
    // Read from AppStorage or preferences
    return { connected: false };
  }
}

// Widget configuration in module.json5
{
  "extensionAbilities": [
    {
      "name": "HelixWidgetAbility",
      "srcEntry": "./ets/widget/HelixWidgetAbility.ets",
      "type": "form",
      "metadata": [
        {
          "name": "ohos.extension.form",
          "resource": "$profile:form_config"
        }
      ]
```

```
    }
  ]
}
```

4.8 HarmonyOS NEXT Specific Considerations

Aspect	HarmonyOS NEXT	Android
App enumeration	Cannot list all installed apps (privacy)	Full access via PackageManager
VPN permission	`MANAGE_VPN` requires ACL (API 12+)	`BIND_VPN_SERVICE` via system dialog
Emulator	ARM simulator; VPN auth component may be missing	Full emulator support (with some limitations)
Native bridge	N-API (Node-API based on Node.js 12.x)	JNI
Build system	DevEco Studio + hvigor	Android Studio + Gradle
Package format	HAP (HarmonyOS Ability Package)	APK / AAB
Distribution	Huawei AppGallery	Google Play, F-Droid, direct
UI framework	ArkUI (ArkTS) + Flutter embedding	Native + Flutter embedding

Key differences from Android VpnService:

1. **No global app enumeration**: HarmonyOS NEXT restricts listing all installed apps, which affects per-app split tunneling UIs. The app must use preset app lists or manual package name entry.
2. **VpnExtensionAbility vs VpnService**: HarmonyOS uses `VpnExtensionAbility` with `onCreate`/`onDestroy` callbacks instead of extending `VpnService`.
3. **N-API vs JNI**: The native bridge uses Node-API (N-API) instead of JNI, though both follow similar patterns for wrapping native objects.
4. **Permission model**: `MANAGE_VPN` was restricted to system apps until API 11; API 12+ enables third-party apps via ACL.

---

5. Mobile UI/UX Design

5.1 Flutter Design System

Reconciliation note (Revision 3): the previous revision of this class hardcoded an independently-invented indigo (`primaryColor = 0xFF6366F1`) as the Material 3 seed color — never reconciled against the canonical, already-built OpenDesign token system (`docs/design/opendesign/helix/ tokens.css`, `docs/design/tokens/ color.json`), and not the color Android/ iOS/HarmonyOS should actually render (`docs/research/CROSS_CUTTING_GAP_ANALYSIS.md` §1.2 finding #1). The values below are corrected to the canonical OpenDesign primary scale (`color.json` `primary.500`/`.600`) and secondary accent (`color.json` `accent.400`); `ColorScheme.fromSeed()` still generates the full Material 3 tonal palette, but now from the real brand seed instead of an invented one. Do not reintroduce a different brand hex here — extend `docs/design/` if a platform-specific adjustment is ever needed.

```
// lib/theme/helix_theme.dart
import 'package:flutter/material.dart';
import 'dart:io' show Platform;

class HelixTheme {
  // Brand colors — sourced from docs/design/tokens/color.json
        `primary`/
  // `accent` scales (canonical OpenDesign teal, not an
        independently
  // invented palette; see the reconciliation note above §5.1's
        code block)
  static const Color primaryColor = Color(0xFF00897B); //
        color.json primary.500
  static const Color primaryDark = Color(0xFF00796B);  //
        color.json primary.600
  static const Color accentColor = Color(0xFF00BCD4);  //
        color.json accent.400
  static const Color errorColor = Color(0xFFD32F2F);   //
        color.json semantic.error (was invented 0xFFEF4444)
  static const Color warningColor = Color(0xFFF57C00); //
        color.json semantic.warning (was invented 0xFFF59E0B)

  // Connection state colors — color.json `semantic` scale
        (reconciled
  // 2026-07-04; previously invented ad-hoc hex values, same class
        of drift
  // the brand-primary teal fix above already closed for
        primaryColor/accentColor)
  static const Color connected = Color(0xFF4CAF50);    //
        color.json semantic.connected
  static const Color connecting = Color(0xFFFF9800);   //
        color.json semantic.connecting
```

```dart
static const Color disconnected = Color(0xFF6B7280); //
      color.json semantic.neutral (intentional: disconnected
      reads as neutral-gray, not alarm-red, in this UX)
static const Color error = Color(0xFFD32F2F);        //
      color.json semantic.error

// Background colors
static const Color darkBackground = Color(0xFF0F172A);
static const Color darkSurface = Color(0xFF1E293B);
static const Color lightBackground = Color(0xFFF8FAFC);
static const Color lightSurface = Colors.white;

// Text colors
static const Color textPrimary = Color(0xFF1E293B);
static const Color textSecondary = Color(0xFF64748B);
static const Color textOnDark = Colors.white;

/// Get theme based on platform and brightness preference
static ThemeData getTheme({required bool isDarkMode}) {
  final baseTheme = isDarkMode ? _darkTheme : _lightTheme;

  if (Platform.isIOS) {
    return baseTheme.copyWith(
      platform: TargetPlatform.iOS,
      pageTransitionsTheme: const PageTransitionsTheme(
        builders: {
          TargetPlatform.iOS: CupertinoPageTransitionsBuilder(),
        },
      ),
    );
  }

  return baseTheme;
}

static final ThemeData _lightTheme = ThemeData(
  useMaterial3: true,
  brightness: Brightness.light,
  colorScheme: ColorScheme.fromSeed(
    seedColor: primaryColor,
    brightness: Brightness.light,
  ),
  scaffoldBackgroundColor: lightBackground,
  cardTheme: CardTheme(
    elevation: 0,
    shape: RoundedRectangleBorder(borderRadius:
      BorderRadius.circular(16)),
    color: lightSurface,
  ),
  appBarTheme: const AppBarTheme(
    elevation: 0,
    centerTitle: true,
```

```
      backgroundColor: lightSurface,
      foregroundColor: textPrimary,
    ),
    bottomNavigationBarTheme: const BottomNavigationBarThemeData(
      backgroundColor: lightSurface,
      selectedItemColor: primaryColor,
      unselectedItemColor: textSecondary,
      type: BottomNavigationBarType.fixed,
    ),
  );

  static final ThemeData _darkTheme = ThemeData(
    useMaterial3: true,
    brightness: Brightness.dark,
    colorScheme: ColorScheme.fromSeed(
      seedColor: primaryColor,
      brightness: Brightness.dark,
    ),
    scaffoldBackgroundColor: darkBackground,
    cardTheme: CardTheme(
      elevation: 0,
      shape: RoundedRectangleBorder(borderRadius:
        BorderRadius.circular(16)),
      color: darkSurface,
    ),
    appBarTheme: const AppBarTheme(
      elevation: 0,
      centerTitle: true,
      backgroundColor: darkSurface,
      foregroundColor: textOnDark,
    ),
    bottomNavigationBarTheme: const BottomNavigationBarThemeData(
      backgroundColor: darkSurface,
      selectedItemColor: primaryColor,
      unselectedItemColor: textSecondary,
      type: BottomNavigationBarType.fixed,
    ),
  );
}
```

5.2 Connection Screen (Main)

```
// lib/ui/screens/connection_screen.dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

class ConnectionScreen extends ConsumerWidget {
  const ConnectionScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
```

```dart
  final connectionState = ref.watch(connectionStateProvider);
  final selectedServer = ref.watch(selectedServerProvider);
  final connectionStats = ref.watch(connectionStatsProvider);
  final isDarkMode = ref.watch(themeProvider);

  return Scaffold(
    body: SafeArea(
      child: Column(
        children: [
          // Header with current server
          _buildServerSelector(context, selectedServer),

          const Spacer(),

          // Connection status and toggle
          _buildConnectionStatus(connectionState),
          const SizedBox(height: 32),

          // Main connection button
          ConnectionButton(
            state: connectionState,
            onToggle: () => _toggleConnection(ref,
      connectionState),
          ),

          const Spacer(),

          // Connection stats (visible when connected)
          if (connectionState == ConnectionState.connected)
            ConnectionStatsPanel(stats: connectionStats),

          const SizedBox(height: 24),
        ],
      ),
    ),
  );
}

Widget _buildServerSelector(BuildContext context, Server?
      server) {
  return Padding(
    padding: const EdgeInsets.all(16),
    child: InkWell(
      onTap: () => Navigator.pushNamed(context, '/servers'),
      borderRadius: BorderRadius.circular(12),
      child: Container(
        padding: const EdgeInsets.symmetric(horizontal: 16,
      vertical: 12),
        decoration: BoxDecoration(
          borderRadius: BorderRadius.circular(12),
          color: Theme.of(context).cardTheme.color,
        ),
```

```
        child: Row(
          children: [
            // Flag icon
            CountryFlag(
              countryCode: server?.countryCode ?? 'auto',
              size: 32,
            ),
            const SizedBox(width: 12),

            // Server info
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    server?.displayName ?? 'Optimal Location',
                    style: const TextStyle(
                      fontSize: 16,
                      fontWeight: FontWeight.w600,
                    ),
                  ),
                  if (server != null)
                    Text(
                      '${server.city} \u00B7 ${server.load}%
      load',
                      style: TextStyle(
                        fontSize: 12,
                        color:
      Theme.of(context).colorScheme.onSurface.withOpacity(0.6),
                      ),
                    ),
                ],
              ),
            ),

            // Latency indicator
            if (server != null)
              _buildLatencyIndicator(server.latencyMs),

            const Icon(Icons.chevron_right),
          ],
        ),
      ),
    ),
  );
}

Widget _buildConnectionStatus(ConnectionState state) {
  String statusText;
  Color statusColor;

  switch (state) {
```

```
    case ConnectionState.connected:
      statusText = 'Connected';
      statusColor = HelixTheme.connected;
    case ConnectionState.connecting:
      statusText = 'Connecting...';
      statusColor = HelixTheme.connecting;
    case ConnectionState.reconnecting:
      statusText = 'Reconnecting...';
      statusColor = HelixTheme.connecting;
    case ConnectionState.killSwitchActive:
      // Fail-closed: all non-VPN traffic is actively blocked at
      the
      // platform firewall layer (`VpnService.setBlocking` / NE
      // `includeRoutes` / HarmonyOS `isBlocking`) — this is
      distinct from
      // "disconnected" and MUST read as an alarming state, not
      a neutral
      // one, per `MVP2_ARCHITECTURE.md` §5.6.
      statusText = 'Blocked · Kill Switch Active';
      statusColor = HelixTheme.error;
    case ConnectionState.disconnecting:
      statusText = 'Disconnecting...';
      statusColor = HelixTheme.warningColor;
    case ConnectionState.disconnected:
      statusText = 'Not Connected';
      statusColor = HelixTheme.disconnected;
    case ConnectionState.connectionFailed:
      statusText = 'Connection Failed';
      statusColor = HelixTheme.error;
  }

  return Column(
    children: [
      // Status dot
      Container(
        width: 8,
        height: 8,
        decoration: BoxDecoration(
          shape: BoxShape.circle,
          color: statusColor,
          boxShadow: [
            BoxShadow(
              color: statusColor.withOpacity(0.4),
              blurRadius: 8,
              spreadRadius: 2,
            ),
          ],
        ),
      ),
      const SizedBox(height: 8),
      Text(
        statusText,
```

```
        style: TextStyle(
          fontSize: 18,
          fontWeight: FontWeight.w600,
          color: statusColor,
        ),
      ),
    ],
  );
}

Widget _buildLatencyIndicator(int latencyMs) {
  Color color;
  if (latencyMs < 50) color = HelixTheme.connected;
  else if (latencyMs < 100) color = HelixTheme.warningColor;
  else color = HelixTheme.error;

  return Container(
    padding: const EdgeInsets.symmetric(horizontal: 8,
      vertical: 4),
    decoration: BoxDecoration(
      color: color.withOpacity(0.1),
      borderRadius: BorderRadius.circular(8),
    ),
    child: Text(
      '${latencyMs}ms',
      style: TextStyle(
        fontSize: 12,
        color: color,
        fontWeight: FontWeight.w500,
      ),
    ),
  );
}

void _toggleConnection(WidgetRef ref, ConnectionState state) {
  switch (state) {
    case ConnectionState.disconnected:
    case ConnectionState.connectionFailed:
      ref.read(connectionStateProvider.notifier).connect();
    case ConnectionState.connected:
    case ConnectionState.killSwitchActive:
      // Kill-switch-active is the engine's
      // `Disconnecting: user forces disconnect (explicit
      override)`
      // transition (`MVP2_ARCHITECTURE.md` §5.6) — the toggle
      button is
      // the "explicit override" the user presses to escape a
      stuck
      // fail-closed state.
      ref.read(connectionStateProvider.notifier).disconnect();
    default:
      // Do nothing while transitioning (connecting /
      reconnecting /
```

```
        // disconnecting)
        break;
    }
  }
}
```

5.3 Connection Button with Animation

```
// lib/ui/widgets/connection_button.dart
import 'package:flutter/material.dart';

class ConnectionButton extends StatefulWidget {
  final ConnectionState state;
  final VoidCallback onToggle;

  const ConnectionButton({
    super.key,
    required this.state,
    required this.onToggle,
  });

  @override
  State<ConnectionButton> createState() =>
        _ConnectionButtonState();
}

class _ConnectionButtonState extends State<ConnectionButton>
    with TickerProviderStateMixin {
  late AnimationController _pulseController;
  late AnimationController _rotateController;
  late Animation<double> _pulseAnimation;

  @override
  void initState() {
    super.initState();

    _pulseController = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 1500),
    )..repeat(reverse: true);

    _rotateController = AnimationController(
      vsync: this,
      duration: const Duration(milliseconds: 2000),
    );

    _pulseAnimation = Tween<double>(begin: 1.0, end:
        1.15).animate(
      CurvedAnimation(
        parent: _pulseController,
        curve: Curves.easeInOut,
```

```
    ),
  );
}

@override
void didUpdateWidget(ConnectionButton oldWidget) {
  super.didUpdateWidget(oldWidget);

  // `reconnecting` reuses the same spinner as `connecting` —
      from the
  // user's perspective both are "please wait, working on it"
      states; the
  // distinct label is carried by `_buildConnectionStatus`
      (§5.2), not by a
  // second animation.
  final isBusy = widget.state == ConnectionState.connecting ||
      widget.state == ConnectionState.reconnecting;

  if (isBusy) {
    _rotateController.repeat();
  } else {
    _rotateController.stop();
  }

  if (widget.state == ConnectionState.connected ||
      widget.state == ConnectionState.killSwitchActive) {
    _pulseController.repeat(reverse: true);
  } else if (!isBusy) {
    _pulseController.stop();
    _pulseController.value = 0;
  }
}

@override
void dispose() {
  _pulseController.dispose();
  _rotateController.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  final isConnected = widget.state == ConnectionState.connected;
  final isKillSwitchActive = widget.state ==
      ConnectionState.killSwitchActive;
  final isConnecting = widget.state ==
      ConnectionState.connecting ||
      widget.state == ConnectionState.reconnecting;
  final buttonColor = isKillSwitchActive
      ? HelixTheme.error
      : (isConnected ? HelixTheme.connected :
      HelixTheme.disconnected);
```

```
return AnimatedBuilder(
  animation: Listenable.merge([_pulseController,
    _rotateController]),
  builder: (context, child) {
    return Transform.scale(
      scale: isConnected ? _pulseAnimation.value : 1.0,
      child: GestureDetector(
        onTap: widget.onToggle,
        child: Container(
          width: 140,
          height: 140,
          decoration: BoxDecoration(
            shape: BoxShape.circle,
            gradient: RadialGradient(
              colors: [
                buttonColor.withOpacity(0.3),
                buttonColor.withOpacity(0.1),
                Colors.transparent,
              ],
              stops: const [0.0, 0.6, 1.0],
            ),
          ),
          child: Center(
            child: Container(
              width: 100,
              height: 100,
              decoration: BoxDecoration(
                shape: BoxShape.circle,
                gradient: LinearGradient(
                  begin: Alignment.topLeft,
                  end: Alignment.bottomRight,
                  colors: [
                    buttonColor,
                    buttonColor.withOpacity(0.8),
                  ],
                ),
                boxShadow: [
                  BoxShadow(
                    color: buttonColor.withOpacity(0.4),
                    blurRadius: 30,
                    spreadRadius: (isConnected ||
    isKillSwitchActive) ? 10 : 0,
                  ),
                ],
              ),
              child: Center(
                child: isConnecting
                    ? RotationTransition(
                        turns: _rotateController,
                        child: const Icon(
                          Icons.sync,
                          color: Colors.white,
```

```dart
                              size: 40,
                            ),
                          )
                        : Icon(
                            isKillSwitchActive
                                ? Icons.gpp_bad // fail-closed,
        blocked
                                : (isConnected
                                    ? Icons.gpp_good // tunnel up
                                    : Icons.power_settings_new),
                            color: Colors.white,
                            size: 48,
                          ),
                  ),
                ),
              ),
            ),
          ),
        );
      },
    );
  }
}

/// Mirrors the single canonical `ConnectionStatus` state machine
        owned by
/// `helix-vpn-engine` (`MVP2_ARCHITECTURE.md` §5.6) verbatim —
        this is the
/// Dart-side rendering target for the `"state"` string returned
        by
/// `getStatus` on every platform's MethodChannel (§2.6, §3.1,
        §4.2). No
/// Flutter-only state (e.g. a bespoke `error`) may be added here;
        a platform
/// UI introducing its own ad-hoc state is an architectural
        contract
/// violation per `MVP2_ARCHITECTURE.md` §5.6.
enum ConnectionState {
  disconnected,
  connecting,
  connected,
  /// Handshake or keepalive lost; attempting to re-establish.
        Distinct from
  /// `connecting` (first connection) for UI messaging purposes.
  reconnecting,
  /// Reconnect exceeded the grace period AND kill switch is
        enabled: all
  /// non-VPN traffic is being actively blocked at the platform
        firewall
  /// layer. See `MVP2_SECURITY_PERFORMANCE.md` §2.
  killSwitchActive,
  disconnecting,
  /// Initial connection attempt failed (timeout / handshake
        error), prior to
```

```dart
  /// ever reaching `connected`. Distinct from `reconnecting`.
  connectionFailed,
}
```

5.4 Server Selection Screen

```dart
// lib/ui/screens/server_selection_screen.dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';

class ServerSelectionScreen extends ConsumerStatefulWidget {
  const ServerSelectionScreen({super.key});

  @override
  ConsumerState<ServerSelectionScreen> createState() =>
        _ServerSelectionScreenState();
}

class _ServerSelectionScreenState extends
        ConsumerState<ServerSelectionScreen> {
  String _searchQuery = '';
  ServerSortMode _sortMode = ServerSortMode.latency;

  @override
  Widget build(BuildContext context) {
    final servers = ref.watch(serverListProvider);
    final selectedServer = ref.watch(selectedServerProvider);
    final favorites = ref.watch(favoriteServersProvider);

    final filteredServers = servers.where((server) {
      final matchesSearch = server.displayName.toLowerCase()
          .contains(_searchQuery.toLowerCase()) ||

        server.city.toLowerCase().contains(_searchQuery.toLowerCase())
        ||

        server.country.toLowerCase().contains(_searchQuery.toLowerCase());
      return matchesSearch;
    }).toList();

    // Sort servers
    filteredServers.sort((a, b) {
      switch (_sortMode) {
        case ServerSortMode.latency:
          return a.latencyMs.compareTo(b.latencyMs);
        case ServerSortMode.load:
          return a.load.compareTo(b.load);
        case ServerSortMode.name:
          return a.displayName.compareTo(b.displayName);
      }
    });
```

```dart
    return Scaffold(
      appBar: AppBar(
        title: const Text('Select Server'),
        actions: [
          PopupMenuButton<ServerSortMode>(
            icon: const Icon(Icons.sort),
            onSelected: (mode) => setState(() => _sortMode =
        mode),
            itemBuilder: (context) => [
              const PopupMenuItem(
                value: ServerSortMode.latency,
                child: Text('Sort by Latency'),
              ),
              const PopupMenuItem(
                value: ServerSortMode.load,
                child: Text('Sort by Load'),
              ),
              const PopupMenuItem(
                value: ServerSortMode.name,
                child: Text('Sort by Name'),
              ),
            ],
          ),
        ],
      ),
      body: Column(
        children: [
          // Search bar
          Padding(
            padding: const EdgeInsets.all(16),
            child: TextField(
              decoration: InputDecoration(
                hintText: 'Search servers...',
                prefixIcon: const Icon(Icons.search),
                border: OutlineInputBorder(
                  borderRadius: BorderRadius.circular(12),
                ),
                filled: true,
              ),
              onChanged: (value) => setState(() => _searchQuery =
        value),
            ),
          ),

          // Quick connect button
          Padding(
            padding: const EdgeInsets.symmetric(horizontal: 16),
            child: ListTile(
              leading: const Icon(Icons.bolt, color:
        HelixTheme.accentColor),
              title: const Text('Optimal Location'),
```

```
            subtitle: const Text('Best performance
        automatically'),
            trailing: selectedServer == null
                ? const Icon(Icons.check_circle, color:
        HelixTheme.accentColor)
                : null,
            onTap: () {

        ref.read(selectedServerProvider.notifier).selectOptimal();
              Navigator.pop(context);
            },
          ),
        ),

        const Divider(),

        // Server list
        Expanded(
          child: ListView.builder(
            itemCount: filteredServers.length,
            itemBuilder: (context, index) {
              final server = filteredServers[index];
              final isSelected = server.id ==
        selectedServer?.id;
              final isFavorite = favorites.contains(server.id);

              return ServerListTile(
                server: server,
                isSelected: isSelected,
                isFavorite: isFavorite,
                onTap: () {

        ref.read(selectedServerProvider.notifier).select(server);
                  Navigator.pop(context);
                },
                onFavoriteToggle: () {
                  ref.read(favoriteServersProvider.notifier)
                      .toggle(server.id);
                },
              );
            },
          ),
        ),
      ],
    ),
  );
  }
}

class ServerListTile extends StatelessWidget {
  final Server server;
  final bool isSelected;
  final bool isFavorite;
```

```dart
final VoidCallback onTap;
final VoidCallback onFavoriteToggle;

const ServerListTile({
  super.key,
  required this.server,
  required this.isSelected,
  required this.isFavorite,
  required this.onTap,
  required this.onFavoriteToggle,
});

@override
Widget build(BuildContext context) {
  Color latencyColor;
  if (server.latencyMs < 50) latencyColor =
      HelixTheme.connected;
  else if (server.latencyMs < 100) latencyColor =
      HelixTheme.warningColor;
  else latencyColor = HelixTheme.error;

  return ListTile(
    leading: Stack(
      children: [
        CountryFlag(countryCode: server.countryCode, size: 36),
        if (isSelected)
          Positioned(
            right: 0,
            bottom: 0,
            child: Container(
              padding: const EdgeInsets.all(2),
              decoration: const BoxDecoration(
                color: HelixTheme.connected,
                shape: BoxShape.circle,
              ),
              child: const Icon(Icons.check, size: 10, color:
      Colors.white),
            ),
          ),
      ],
    ),
    title: Text(server.displayName),
    subtitle: Row(
      children: [
        Container(
          width: 6,
          height: 6,
          decoration: BoxDecoration(
            shape: BoxShape.circle,
            color: latencyColor,
          ),
        ),
```

```dart
          const SizedBox(width: 4),
          Text('${server.latencyMs}ms'),
          const SizedBox(width: 12),
          Text('${server.load}% load'),
        ],
      ),
      trailing: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          // Protocol support badges. `ProtocolType.OpenVPN` is a
        RESERVED,
          // unimplemented placeholder in `helix-core`
        (`MVP2_SHARED_CORE.md`
          // §2.3) — it MUST NOT appear here or anywhere else a
        user could
          // select it as an available protocol.
          if (server.supportsWireGuard)
            _buildProtocolBadge('WG'),
          if (server.supportsShadowsocks)
            _buildProtocolBadge('SS'),
          if (server.supportsMasque)
            _buildProtocolBadge('MQ'),
          if (server.supportsMultiHop)
            _buildProtocolBadge('MH'),
          const SizedBox(width: 8),

          // Favorite toggle
          IconButton(
            icon: Icon(
              isFavorite ? Icons.star : Icons.star_border,
              color: isFavorite ? HelixTheme.warningColor : null,
            ),
            onPressed: onFavoriteToggle,
          ),
        ],
      ),
      onTap: onTap,
      selected: isSelected,
    );
  }

  Widget _buildProtocolBadge(String label) {
    return Container(
      margin: const EdgeInsets.only(right: 4),
      padding: const EdgeInsets.symmetric(horizontal: 4,
        vertical: 2),
      decoration: BoxDecoration(
        color: HelixTheme.primaryColor.withOpacity(0.1),
        borderRadius: BorderRadius.circular(4),
      ),
      child: Text(
        label,
```

```
          style: const TextStyle(
            fontSize: 10,
            color: HelixTheme.primaryColor,
            fontWeight: FontWeight.w500,
          ),
        ),
      );
    }
}

enum ServerSortMode { latency, load, name }
```

5.5 Settings Screen

```
// lib/ui/screens/settings_screen.dart
class SettingsScreen extends ConsumerWidget {
  const SettingsScreen({super.key});

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final settings = ref.watch(appSettingsProvider);

    return Scaffold(
      appBar: AppBar(title: const Text('Settings')),
      body: ListView(
        children: [
          // Connection Settings
          _buildSectionHeader(context, 'Connection'),

          ListTile(
            leading: const Icon(Icons.security),
            title: const Text('Protocol'),
            subtitle: Text(settings.protocol.displayName),
            trailing: const Icon(Icons.chevron_right),
            onTap: () => _showProtocolSelector(context, ref),
          ),

          ListTile(
            leading: const Icon(Icons.call_split),
            title: const Text('Split Tunneling'),
            subtitle: Text(settings.splitTunnelEnabled ?
        'Enabled' : 'Disabled'),
            trailing: const Icon(Icons.chevron_right),
            onTap: () => Navigator.pushNamed(context, '/
        split_tunneling'),
          ),

          ListTile(
            leading: const Icon(Icons.shield),
            title: const Text('Kill Switch'),
            subtitle: Text(settings.killSwitchEnabled ?
        'Enabled' : 'Disabled'),
```

```
    trailing: Switch(
      value: settings.killSwitchEnabled,
      onChanged: (value) {

ref.read(appSettingsProvider.notifier).setKillSwitch(value);
      },
    ),
  ),

  // Auto-Connect Settings
  _buildSectionHeader(context, 'Auto-Connect'),

  ListTile(
    leading: const Icon(Icons.wifi),
    title: const Text('Connect on Untrusted Wi-Fi'),
    trailing: Switch(
      value: settings.connectOnUntrustedWifi,
      onChanged: (value) {

ref.read(appSettingsProvider.notifier).setConnectOnUntrustedWifi(value);
      },
    ),
  ),

  ListTile(
    leading: const Icon(Icons.signal_cellular_alt),
    title: const Text('Connect on Cellular'),
    trailing: Switch(
      value: settings.connectOnCellular,
      onChanged: (value) {

ref.read(appSettingsProvider.notifier).setConnectOnCellular(value);
      },
    ),
  ),

  ListTile(
    leading: const Icon(Icons.network_check),
    title: const Text('Trusted Wi-Fi Networks'),
    subtitle:
Text('${settings.trustedWifiNetworks.length} networks'),
    trailing: const Icon(Icons.chevron_right),
    onTap: () => Navigator.pushNamed(context, '/
trusted_wifi'),
  ),

  // Security Settings
  _buildSectionHeader(context, 'Security'),

  ListTile(
    leading: const Icon(Icons.fingerprint),
    title: const Text('Biometric Authentication'),
```

```
    subtitle: Text(settings.biometricAuth ? 'Enabled' :
'Disabled'),
    trailing: Switch(
      value: settings.biometricAuth,
      onChanged: (value) => _toggleBiometric(context,
ref, value),
    ),
  ),

  ListTile(
    leading: const Icon(Icons.dns),
    title: const Text('Custom DNS'),
    subtitle: Text(settings.customDns?.join(', ') ??
'Default'),
    trailing: const Icon(Icons.chevron_right),
    onTap: () => _showDnsConfig(context, ref),
  ),

  // App Settings
  _buildSectionHeader(context, 'App'),

  ListTile(
    leading: const Icon(Icons.dark_mode),
    title: const Text('Dark Mode'),
    trailing: DropdownButton<ThemeMode>(
      value: settings.themeMode,
      underline: const SizedBox.shrink(),
      items: ThemeMode.values.map((mode) {
        return DropdownMenuItem(
          value: mode,
          child: Text(mode.displayName),
        );
      }).toList(),
      onChanged: (mode) {
        if (mode != null) {

ref.read(appSettingsProvider.notifier).setThemeMode(mode);
        }
      },
    ),
  ),

  ListTile(
    leading: const Icon(Icons.vibration),
    title: const Text('Haptic Feedback'),
    trailing: Switch(
      value: settings.hapticFeedback,
      onChanged: (value) {

ref.read(appSettingsProvider.notifier).setHapticFeedback(value);
      },
    ),
  ),
```

```
          const AboutListTile(
            icon: Icon(Icons.info),
            applicationName: 'Helix VPN',
            applicationVersion: '1.0.0',
            applicationLegalese: '\u00A9 2025 Helix VPN Inc.',
          ),
        ],
      ),
    );
  }

  Widget _buildSectionHeader(BuildContext context, String title) {
    return Padding(
      padding: const EdgeInsets.fromLTRB(16, 24, 16, 8),
      child: Text(
        title.toUpperCase(),
        style: TextStyle(
          fontSize: 12,
          fontWeight: FontWeight.w600,
          color: Theme.of(context).colorScheme.primary,
          letterSpacing: 1,
        ),
      ),
    );
  }
}
```

5.6 Split Tunneling Configuration Screen

```
// lib/ui/screens/split_tunneling_screen.dart
class SplitTunnelingScreen extends ConsumerStatefulWidget {
  const SplitTunnelingScreen({super.key});

  @override
  ConsumerState<SplitTunnelingScreen> createState() =>
        _SplitTunnelingScreenState();
}

class _SplitTunnelingScreenState extends
        ConsumerState<SplitTunnelingScreen>
    with SingleTickerProviderStateMixin {
  late TabController _tabController;
  SplitTunnelMode _mode = SplitTunnelMode.off;
  final Set<String> _selectedApps = {};
  List<InstalledApp> _installedApps = [];
  bool _isLoading = true;

  @override
  void initState() {
    super.initState();
```

```dart
    _tabController = TabController(length: 3, vsync: this);
    _loadApps();
  }

  Future<void> _loadApps() async {
    final apps = await
        ref.read(vpnServiceProvider).getInstalledApps();
    setState(() {
      _installedApps = apps;
      _isLoading = false;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Split Tunneling'),
        bottom: TabBar(
          controller: _tabController,
          tabs: const [
            Tab(text: 'Off'),
            Tab(text: 'Include'),
            Tab(text: 'Exclude'),
          ],
          onTap: (index) {
            setState(() {
              _mode = SplitTunnelMode.values[index];
            });
          },
        ),
      ),
      body: _isLoading
          ? const Center(child: CircularProgressIndicator())
          : Column(
              children: [
                // Mode explanation
                Padding(
                  padding: const EdgeInsets.all(16),
                  child: _buildModeExplanation(),
                ),

                // App selection list
                Expanded(
                  child: ListView.builder(
                    itemCount: _installedApps.length,
                    itemBuilder: (context, index) {
                      final app = _installedApps[index];
                      final isSelected =
        _selectedApps.contains(app.packageName);

                      return CheckboxListTile(
```

```
                      secondary: app.icon != null
                          ? Image.memory(app.icon!, width: 40,
      height: 40)
                          : const Icon(Icons.android, size: 40),
                      title: Text(app.name),
                      subtitle: Text(app.packageName),
                      value: isSelected,
                      onChanged: _mode == SplitTunnelMode.off
                          ? null
                          : (value) {
                              setState(() {
                                if (value == true) {

      _selectedApps.add(app.packageName);
                                } else {

      _selectedApps.remove(app.packageName);
                                }
                              });
                            },
                    );
                  },
                ),
              ),

              // Save button
              Padding(
                padding: const EdgeInsets.all(16),
                child: SizedBox(
                  width: double.infinity,
                  child: FilledButton(
                    onPressed: () => _saveConfiguration(),
                    child: const Text('Save Configuration'),
                  ),
                ),
              ),
            ],
          ),
    );
  }

  Widget _buildModeExplanation() {
    String explanation;
    switch (_mode) {
      case SplitTunnelMode.off:
        explanation = 'All app traffic will be routed through the
        VPN.';
      case SplitTunnelMode.include:
        explanation = 'Only selected apps will use the VPN. All
        other apps will connect directly.';
      case SplitTunnelMode.exclude:
        explanation = 'Selected apps will bypass the VPN. All
        other apps will use the VPN.';
```

```
    }

    return Container(
      padding: const EdgeInsets.all(12),
      decoration: BoxDecoration(
        color:
        Theme.of(context).colorScheme.primary.withOpacity(0.1),
        borderRadius: BorderRadius.circular(8),
      ),
      child: Row(
        children: [
          Icon(Icons.info_outline,
              color: Theme.of(context).colorScheme.primary),
          const SizedBox(width: 12),
          Expanded(child: Text(explanation)),
        ],
      ),
    );
  }

  void _saveConfiguration() {
    ref.read(vpnServiceProvider).setSplitTunneling(
      mode: _mode,
      apps: _selectedApps.toList(),
    );
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('Split tunneling configuration
        saved')),
    );
  }
}

enum SplitTunnelMode { off, include, exclude }
```

5.7 Notification Design

```
// lib/services/notification_service.dart
import 'package:flutter_local_notifications/
        flutter_local_notifications.dart';

class NotificationService {
  static final NotificationService _instance =
        NotificationService._internal();
  factory NotificationService() => _instance;
  NotificationService._internal();

  final FlutterLocalNotificationsPlugin _notifications =
      FlutterLocalNotificationsPlugin();

  Future<void> initialize() async {
```

```dart
  const androidSettings =
      AndroidInitializationSettings('@drawable/
      ic_notification');
  const iosSettings = DarwinInitializationSettings(
    requestAlertPermission: false,
    requestBadgePermission: false,
    requestSoundPermission: false,
  );

  const initSettings = InitializationSettings(
    android: androidSettings,
    iOS: iosSettings,
  );

  await _notifications.initialize(initSettings);
}

// Android: Foreground service notification (handled natively)
// This is for Flutter-triggered notifications only

Future<void> showConnectionNotification({
  required String serverLocation,
  required String protocol,
}) async {
  const androidDetails = AndroidNotificationDetails(
    'vpn_status',
    'VPN Connection Status',
    channelDescription: 'Shows when Helix VPN is connected',
    importance: Importance.low,
    priority: Priority.low,
    ongoing: true,
    autoCancel: false,
    showWhen: false,
  );

  const iosDetails = DarwinNotificationDetails(
    presentAlert: false,
    presentBadge: false,
    presentSound: false,
  );

  const details = NotificationDetails(
    android: androidDetails,
    iOS: iosDetails,
  );

  await _notifications.show(
    1,
    'Helix VPN Connected',
    'Server: $serverLocation \u00B7 $protocol',
    details,
  );
```

```dart
  }

  Future<void> showReconnectionWarning() async {
    const androidDetails = AndroidNotificationDetails(
      'vpn_alerts',
      'VPN Alerts',
      channelDescription: 'Important VPN connection alerts',
      importance: Importance.high,
      priority: Priority.high,
    );

    await _notifications.show(
      2,
      'Connection Unstable',
      'Helix VPN is reconnecting to maintain your secure
        connection.',
      const NotificationDetails(android: androidDetails),
    );
  }

  Future<void> cancelAll() async {
    await _notifications.cancelAll();
  }
}
```

5.8 Widget Support

Android Home Screen Widget:

```xml
<!-- android/app/src/main/res/xml/vpn_widget_info.xml -->
<appwidget-provider xmlns:android="http://schemas.android.com/apk/
        res/android"
    android:minWidth="180dp"
    android:minHeight="40dp"
    android:updatePeriodMillis="0"
    android:initialLayout="@layout/vpn_widget"
    android:previewImage="@drawable/widget_preview"
    android:widgetCategory="home_screen"
    android:resizeMode="horizontal" />

// VpnAppWidgetProvider.kt
class VpnAppWidgetProvider : AppWidgetProvider() {
    override fun onUpdate(
        context: Context,
        appWidgetManager: AppWidgetManager,
        appWidgetIds: IntArray
    ) {
        for (appWidgetId in appWidgetIds) {
            updateWidget(context, appWidgetManager, appWidgetId)
        }
    }
```

```kotlin
private fun updateWidget(
    context: Context,
    appWidgetManager: AppWidgetManager,
    appWidgetId: Int
) {
    val views = RemoteViews(context.packageName,
    R.layout.vpn_widget)

    val isConnected = HelixVpnService.isRunning

    views.setTextViewText(
        R.id.widget_status,
        if (isConnected) "Connected" else "Disconnected"
    )
    views.setImageViewResource(
        R.id.widget_icon,
        if (isConnected) R.drawable.ic_widget_on else
    R.drawable.ic_widget_off
    )

    // Toggle intent
    val toggleIntent = PendingIntent.getBroadcast(
        context,
        0,
        Intent(context,
    VpnAppWidgetProvider::class.java).apply {
            action = "TOGGLE_VPN"
        },
        PendingIntent.FLAG_UPDATE_CURRENT or
    PendingIntent.FLAG_IMMUTABLE
    )
    views.setOnClickPendingIntent(R.id.widget_button,
    toggleIntent)

    appWidgetManager.updateAppWidget(appWidgetId, views)
}

override fun onReceive(context: Context, intent: Intent) {
    super.onReceive(context, intent)

    if (intent.action == "TOGGLE_VPN") {
        if (HelixVpnService.isRunning) {
            context.startService(
                Intent(context, HelixVpnService::class.java)
                    .setAction(HelixVpnService.ACTION_DISCONNECT)
            )
        } else {
            // Launch app for connection
            context.startActivity(

    context.packageManager.getLaunchIntentForPackage(context.packageName)
            )
        }
```

```
        }
    }
}
```

iOS Widget (using SwiftUI):

```swift
// HelixVpnWidget/HelixVpnWidget.swift
import WidgetKit
import SwiftUI

struct HelixVpnWidgetEntry: TimelineEntry {
    let date: Date
    let isConnected: Bool
    let serverLocation: String
}

struct HelixVpnWidgetProvider: TimelineProvider {
    func placeholder(in context: Context) -> HelixVpnWidgetEntry {
        HelixVpnWidgetEntry(date: Date(), isConnected: false,
        serverLocation: "Not Connected")
    }

    func getSnapshot(in context: Context, completion: @escaping
        (HelixVpnWidgetEntry) -> Void) {
        let entry = loadCurrentState()
        completion(entry)
    }

    func getTimeline(in context: Context, completion: @escaping
        (Timeline<HelixVpnWidgetEntry>) -> Void) {
        let entry = loadCurrentState()
        let timeline = Timeline(entries: [entry],
        policy: .after(Date().addingTimeInterval(60)))
        completion(timeline)
    }

    private func loadCurrentState() -> HelixVpnWidgetEntry {
        let sharedDefaults = UserDefaults(suiteName:
        "group.com.helix.vpn")
        let isConnected = sharedDefaults?.string(forKey:
        "tunnel_state") == "connected"
        let serverLocation = sharedDefaults?.string(forKey:
        "last_server") ?? "Not Connected"

        return HelixVpnWidgetEntry(
            date: Date(),
            isConnected: isConnected,
            serverLocation: serverLocation
        )
    }
}

struct HelixVpnWidgetView: View {
```

```swift
    var entry: HelixVpnWidgetProvider.Entry
    @Environment(\.widgetFamily) var family

    var body: some View {
        VStack(spacing: 8) {
            Image(systemName: entry.isConnected ?
"shield.checkered" : "shield")
                .font(.title2)
                .foregroundColor(entry.isConnected ? .green : .gray)

            Text(entry.isConnected ? "Connected" : "Disconnected")
                .font(.caption)
                .fontWeight(.semibold)

            if entry.isConnected {
                Text(entry.serverLocation)
                    .font(.caption2)
                    .foregroundColor(.secondary)
                    .lineLimit(1)
            }
        }
        .padding()
        .containerBackground(.fill.tertiary, for: .widget)
    }
}

@main
struct HelixVpnWidget: Widget {
    let kind: String = "HelixVpnWidget"

    var body: some WidgetConfiguration {
        StaticConfiguration(kind: kind, provider:
HelixVpnWidgetProvider()) { entry in
            HelixVpnWidgetView(entry: entry)
        }
        .configurationDisplayName("Helix VPN Status")
        .description("Quickly see your VPN connection status.")
        .supportedFamilies([.systemSmall, .systemMedium])
    }
}
```

---

6. Mobile-Specific Features

6.1 Biometric Authentication

```dart
// lib/services/biometric_service.dart
import 'package:local_auth/local_auth.dart';
import 'package:local_auth_android/local_auth_android.dart';
import 'package:local_auth_darwin/local_auth_darwin.dart';
```

```dart
class BiometricService {
  static final LocalAuthentication _localAuth =
        LocalAuthentication();

  /// Check if biometric authentication is available
  static Future<bool> isAvailable() async {
    final isDeviceSupported = await
        _localAuth.isDeviceSupported();
    final canCheck = await _localAuth.canCheckBiometrics;
    return isDeviceSupported && canCheck;
  }

  /// Get available biometric types
  static Future<List<BiometricType>> getAvailableTypes() async {
    return await _localAuth.getAvailableBiometrics();
  }

  /// Authenticate user with biometrics
  static Future<bool> authenticate({
    String localizedReason = 'Authenticate to access VPN',
    bool useErrorDialogs = true,
    bool stickyAuth = false,
    bool sensitiveTransaction = true,
  }) async {
    try {
      return await _localAuth.authenticate(
        localizedReason: localizedReason,
        authMessages: const [
          AndroidAuthMessages(
            signInTitle: 'Helix VPN Authentication',
            cancelButton: 'Cancel',
            biometricHint: 'Verify your identity',
            biometricNotRecognized:
        'Biometric not recognized, try again',
            biometricRequiredTitle: 'Biometric authentication
        required',
            deviceCredentialsRequiredTitle: 'Device credentials
        required',
            deviceCredentialsSetupDescription: 'Please set up
        device credentials',
            goToSettingsButton: 'Go to Settings',
            goToSettingsDescription: 'Please set up biometric
        authentication in Settings',
          ),
          IOSAuthMessages(
            cancelButton: 'Cancel',
            goToSettingsButton: 'Go to Settings',
            goToSettingsDescription: 'Please set up biometric
        authentication in Settings',
            lockOut: 'Biometric authentication is locked out',
          ),
        ],
```

```
        options: AuthenticationOptions(
          useErrorDialogs: useErrorDialogs,
          stickyAuth: stickyAuth,
          sensitiveTransaction: sensitiveTransaction,
          biometricOnly: false, // Allow fallback to PIN/password
        ),
      );
    } catch (e) {
      return false;
    }
  }

  /// Authenticate before VPN connection
  static Future<bool> authenticateForConnection() async {
    return authenticate(
      localizedReason: 'Authenticate to connect Helix VPN',
      sensitiveTransaction: true,
    );
  }

  /// Authenticate before accessing sensitive settings
  static Future<bool> authenticateForSettings() async {
    return authenticate(
      localizedReason: 'Authenticate to access VPN settings',
      sensitiveTransaction: true,
    );
  }
}
```

6.2 Auto-Connect on Untrusted Wi-Fi

```
// lib/services/auto_connect_service.dart
import 'package:connectivity_plus/connectivity_plus.dart';
import 'package:wifi_info_flutter/wifi_info_flutter.dart';

class AutoConnectService {
  final Ref ref;
  final Connectivity _connectivity = Connectivity();
  StreamSubscription? _connectivitySubscription;

  AutoConnectService(this.ref);

  void startMonitoring() {
    _connectivitySubscription =
        _connectivity.onConnectivityChanged
        .listen(_handleConnectivityChange);
  }

  void stopMonitoring() {
    _connectivitySubscription?.cancel();
  }
```

```dart
  Future<void> _handleConnectivityChange(ConnectivityResult
        result) async {
    final settings = ref.read(appSettingsProvider);
    if (!settings.autoConnectEnabled) return;

    if (result == ConnectivityResult.wifi) {
      final wifiName = await WifiInfo().getWifiName();
      final isTrusted =
        settings.trustedWifiNetworks.contains(wifiName);

      if (!isTrusted && settings.connectOnUntrustedWifi) {
        // Auto-connect on untrusted Wi-Fi
        await
        ref.read(connectionStateProvider.notifier).connect();
      } else if (isTrusted && settings.disconnectOnTrustedWifi) {
        // Disconnect on trusted Wi-Fi
        await
        ref.read(connectionStateProvider.notifier).disconnect();
      }
    } else if (result == ConnectivityResult.mobile) {
      if (settings.connectOnCellular) {
        await
        ref.read(connectionStateProvider.notifier).connect();
      }
    }
  }
}
```

6.3 Siri Shortcuts (iOS) / App Actions (Android)

iOS Siri Shortcuts:

```swift
// Runner/AppDelegate.swift additions
import Intents

// Register Siri intents for VPN control
class IntentHandler: INExtension {
    override func handler(for intent: INIntent) -> Any {
        if intent is ConnectVpnIntent {
            return ConnectVpnIntentHandler()
        } else if intent is DisconnectVpnIntent {
            return DisconnectVpnIntentHandler()
        }
        return self
    }
}

class ConnectVpnIntentHandler: NSObject, ConnectVpnIntentHandling
        {
    func handle(intent: ConnectVpnIntent, completion: @escaping
        (ConnectVpnIntentResponse) -> Void) {
```

```swift
        // Trigger VPN connection
        VpnConfigurationManager.shared.loadOrCreateConfiguration
        { manager, error in
            guard let manager = manager else {
        
        completion(ConnectVpnIntentResponse(code: .failure,
        userActivity: nil))
                return
            }
            VpnConfigurationManager.shared.connect(manager:
        manager, config: defaultConfig) { error in
                let response = ConnectVpnIntentResponse(
                    code: error == nil ? .success : .failure,
                    userActivity: nil
                )
                completion(response)
            }
        }
    }
}
```

Android App Actions:

```xml
<!-- android/app/src/main/res/xml/actions.xml -->
<actions>
    <action intentName="actions.intent.OPEN_APP_FEATURE">
        <fulfillment
            fulfillmentMode="actions.fulfillment.DEEPLINK"
            urlTemplate="helixvpn://connect" />
        <fulfillment
            fulfillmentMode="actions.fulfillment.DEEPLINK"
            urlTemplate="helixvpn://disconnect" />
        <parameter name="feature">
            <entity-set-reference
        entitySetId="VpnFeatureEntitySet" />
        </parameter>
    </action>
</actions>
```

6.4 Connection Stats and Data Usage

```dart
// lib/models/connection_stats.dart
import 'package:freezed_annotation/freezed_annotation.dart';

part 'connection_stats.freezed.dart';
part 'connection_stats.g.dart';

@freezed
class ConnectionStats with _$ConnectionStats {
  const factory ConnectionStats({
    @Default(0) double uploadSpeed,
    @Default(0) double downloadSpeed,
```

```dart
    @Default(0) int totalUpload,
    @Default(0) int totalDownload,
    @Default(0) int connectionDuration,
    String? serverLocation,
    String? protocol,
    @Default(0) int pingMs,
    @Default([]) List<DataUsagePoint> usageHistory,
  }) = _ConnectionStats;

  factory ConnectionStats.fromJson(Map<String, dynamic> json) =>
      _$ConnectionStatsFromJson(json);
}

@freezed
class DataUsagePoint with _$DataUsagePoint {
  const factory DataUsagePoint({
    required DateTime timestamp,
    required double uploadSpeed,
    required double downloadSpeed,
  }) = _DataUsagePoint;

  factory DataUsagePoint.fromJson(Map<String, dynamic> json) =>
      _$DataUsagePointFromJson(json);
}

// lib/ui/widgets/connection_stats_panel.dart
class ConnectionStatsPanel extends StatelessWidget {
  final ConnectionStats stats;

  const ConnectionStatsPanel({super.key, required this.stats});

  @override
  Widget build(BuildContext context) {
    return Container(
      margin: const EdgeInsets.symmetric(horizontal: 16),
      padding: const EdgeInsets.all(16),
      decoration: BoxDecoration(
        color: Theme.of(context).cardTheme.color,
        borderRadius: BorderRadius.circular(16),
      ),
      child: Column(
        children: [
          // Speed indicators
          Row(
            mainAxisAlignment: MainAxisAlignment.spaceEvenly,
            children: [
              _buildSpeedIndicator(
                icon: Icons.arrow_downward,
                label: 'Download',
                speed: stats.downloadSpeed,
                color: Colors.blue,
              ),
```

```
            Container(height: 40, width: 1, color:
        Colors.grey.shade300),
            _buildSpeedIndicator(
              icon: Icons.arrow_upward,
              label: 'Upload',
              speed: stats.uploadSpeed,
              color: Colors.green,
            ),
          ],
        ),

        const Divider(height: 24),

        // Data usage and duration
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          children: [
            _buildInfoItem('Total Down',
        _formatBytes(stats.totalDownload)),
            _buildInfoItem('Total Up',
        _formatBytes(stats.totalUpload)),
            _buildInfoItem('Duration',
        _formatDuration(stats.connectionDuration)),
            _buildInfoItem('Ping', '${stats.pingMs}ms'),
          ],
        ),

        // Mini speed chart
        if (stats.usageHistory.isNotEmpty)
          SizedBox(
            height: 60,
            child: _buildMiniChart(stats.usageHistory),
          ),
      ],
    ),
  );
}

Widget _buildSpeedIndicator({
  required IconData icon,
  required String label,
  required double speed,
  required Color color,
}) {
  return Column(
    children: [
      Icon(icon, color: color, size: 20),
      const SizedBox(height: 4),
      Text(
        _formatSpeed(speed),
        style: const TextStyle(fontSize: 18, fontWeight:
        FontWeight.bold),
      ),
```

```dart
        Text(label, style: TextStyle(fontSize: 12, color:
        Colors.grey.shade600)),
      ],
    );
  }

  Widget _buildInfoItem(String label, String value) {
    return Column(
      children: [
        Text(value, style: const TextStyle(fontWeight:
        FontWeight.w600)),
        Text(label, style: TextStyle(fontSize: 11, color:
        Colors.grey.shade600)),
      ],
    );
  }

  Widget _buildMiniChart(List<DataUsagePoint> history) {
    // Simple sparkline implementation
    final maxSpeed = history.fold<double>(0, (max, p) =>
        p.downloadSpeed > max ? p.downloadSpeed : max);

    return CustomPaint(
      size: const Size(double.infinity, 60),
      painter: SpeedChartPainter(
        history: history,
        maxSpeed: maxSpeed > 0 ? maxSpeed : 1,
      ),
    );
  }

  String _formatSpeed(double bytesPerSecond) {
    if (bytesPerSecond < 1024) return '$
        {bytesPerSecond.toStringAsFixed(0)} B/s';
    if (bytesPerSecond < 1024 * 1024) return '${(bytesPerSecond /
        1024).toStringAsFixed(1)} KB/s';
    return
        '${(bytesPerSecond / (1024 * 1024)).toStringAsFixed(1)}
        MB/s';
  }

  String _formatBytes(int bytes) {
    if (bytes < 1024) return '$bytes B';
    if (bytes < 1024 * 1024) return '${(bytes /
        1024).toStringAsFixed(1)} KB';
    if (bytes < 1024 * 1024 * 1024) return '${(bytes / (1024 *
        1024)).toStringAsFixed(1)} MB';
    return '${(bytes / (1024 * 1024 * 1024)).toStringAsFixed(2)}
        GB';
  }

  String _formatDuration(int seconds) {
    final duration = Duration(seconds: seconds);
```

```dart
    final hours = duration.inHours;
    final minutes = duration.inMinutes % 60;
    final secs = duration.inSeconds % 60;
    return '${hours.toString().padLeft(2, '0')}:$
        {minutes.toString().padLeft(2, '0')}:$
        {secs.toString().padLeft(2, '0')}';
  }
}
```

6.5 Haptic Feedback

```dart
// lib/services/haptic_service.dart
import 'package:flutter/services.dart';

class HapticService {
  /// Light impact for UI interactions (button taps, toggles)
  static Future<void> lightImpact() async {
    await HapticFeedback.lightImpact();
  }

  /// Medium impact for significant actions
  static Future<void> mediumImpact() async {
    await HapticFeedback.mediumImpact();
  }

  /// Heavy impact for critical actions (connect/disconnect)
  static Future<void> heavyImpact() async {
    await HapticFeedback.heavyImpact();
  }

  /// Success feedback for completed operations
  static Future<void> success() async {
    await HapticFeedback.heavyImpact();
  }

  /// Error feedback for failures
  static Future<void> error() async {
    await HapticFeedback.vibrate();
  }

  /// Connection state change feedback
  static Future<void> connectionStateChanged(bool connected)
        async {
    if (connected) {
      await HapticFeedback.heavyImpact();
      await Future.delayed(const Duration(milliseconds: 100));
      await HapticFeedback.lightImpact();
    } else {
      await HapticFeedback.mediumImpact();
    }
```

```
  }
}
```

6.6 Deep Linking

```dart
// lib/services/deep_link_service.dart
import 'package:go_router/go_router.dart';

class DeepLinkService {
  static final GoRouter router = GoRouter(
    initialLocation: '/',
    routes: [
      GoRoute(
        path: '/',
        builder: (context, state) => const ConnectionScreen(),
      ),
      GoRoute(
        path: '/connect',
        builder: (context, state) {
          // Handle connect deep link
          WidgetsBinding.instance.addPostFrameCallback((_) {

        context.read(connectionStateProvider.notifier).connect();
          });
          return const ConnectionScreen();
        },
      ),
      GoRoute(
        path: '/disconnect',
        builder: (context, state) {
          WidgetsBinding.instance.addPostFrameCallback((_) {

        context.read(connectionStateProvider.notifier).disconnect();
          });
          return const ConnectionScreen();
        },
      ),
      GoRoute(
        path: '/servers',
        builder: (context, state) => const
        ServerSelectionScreen(),
      ),
      GoRoute(
        path: '/settings',
        builder: (context, state) => const SettingsScreen(),
      ),
      GoRoute(
        path: '/settings/split_tunneling',
        builder: (context, state) => const SplitTunnelingScreen(),
      ),
    ],
```

```
  );
}
```

Android manifest deep link configuration:

```xml
<!-- AndroidManifest.xml -->
<activity
    android:name=".MainActivity"
    android:exported="true"
    android:launchMode="singleTop">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category
        android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
    <intent-filter>
        <action android:name="android.intent.action.VIEW" />
        <category android:name="android.intent.category.DEFAULT" /
        >
        <category
        android:name="android.intent.category.BROWSABLE" />
        <data android:scheme="helixvpn" />
    </intent-filter>
</activity>
```

iOS URL scheme configuration:

```xml
<!-- Runner/Info.plist -->
<key>CFBundleURLTypes</key>
<array>
    <dict>
        <key>CFBundleURLName</key>
        <string>com.helix.vpn</string>
        <key>CFBundleURLSchemes</key>
        <array>
            <string>helixvpn</string>
        </array>
    </dict>
</array>
```

6.7 Share Extension (iOS)

```swift
// ShareExtension/ShareViewController.swift
import UIKit
import Social

class ShareViewController: SLComposeServiceViewController {

    override func isContentValid() -> Bool {
        return true
    }
```

```
    override func didSelectPost() {
        // Handle shared content - e.g., import WireGuard config
        from file
        if let item = extensionContext?.inputItems.first as?
        NSExtensionItem,
           let attachments = item.attachments {

            for attachment in attachments {
                if
        attachment.hasItemConformingToTypeIdentifier("public.data")
        {
                    attachment.loadItem(forTypeIdentifier:
        "public.data") {
                        (data, error) in
                        if let url = data as? URL,
                           let configData = try? Data(contentsOf:
        url) {
                            // Save to App Group for main app to
        pick up
                            self.saveSharedConfig(configData,
        filename: url.lastPathComponent)
                        }
                    }
                }
            }
        }

        extensionContext?.completeRequest(returningItems: nil)
    }

    private func saveSharedConfig(_ data: Data, filename: String)
        {
        guard let containerURL = FileManager.default
            .containerURL(forSecurityApplicationGroupIdentifier:
        "group.com.helix.vpn") else {
            return
        }

        let configURL =
        containerURL.appendingPathComponent("shared_configs")
        try? FileManager.default.createDirectory(at: configURL,
        withIntermediateDirectories: true)

        let fileURL = configURL.appendingPathComponent(filename)
        try? data.write(to: fileURL)
    }

    override func configurationItems() -> [Any]! {
        return []
    }
}
```

7. Build & Distribution

7.1 Android Build Configuration

Gradle Flavors:

```
// android/app/build.gradle
android {
    flavorDimensions += "distribution"

    productFlavors {
        playStore {
            dimension "distribution"
            applicationIdSuffix ".play"
            resValue "string", "app_name", "Helix VPN"
        }
        fdroid {
            dimension "distribution"
            applicationIdSuffix ".fdroid"
            resValue "string", "app_name", "Helix VPN (F-Droid)"
            // No Google Play Services dependencies
        }
        direct {
            dimension "distribution"
            resValue "string", "app_name", "Helix VPN"
        }
    }
}
```

Build commands:

```
# Google Play Store (AAB)
flutter build appbundle --release --flavor playStore

# F-Droid (APK per ABI)
flutter build apk --release --flavor fdroid --split-per-abi

# Direct distribution (APK)
flutter build apk --release --flavor direct

# Debug build
flutter run --flavor direct --debug
```

7.2 iOS Build Configuration

```
# Build iOS archive for App Store
flutter build ipa --release --export-options-plist=ios/
        ExportOptions.plist

# Build for TestFlight
flutter build ipa --release --export-method=app-store-connect
```

```bash
# Build for development device
flutter run --release

# Build for simulator (VPN functionality won't work)
flutter build ios --simulator
```

ExportOptions.plist:

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN"
  "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>method</key>
    <string>app-store-connect</string>
    <key>teamID</key>
    <string>YOUR_TEAM_ID</string>
    <key>uploadSymbols</key>
    <true/>
    <key>uploadBitcode</key>
    <false/>
    <key>signingStyle</key>
    <string>manual</string>
    <key>signingCertificate</key>
    <string>Apple Distribution</string>
    <key>provisioningProfiles</key>
    <dict>
        <key>com.helix.vpn</key>
        <string>Helix VPN App Store</string>
        <key>com.helix.vpn.packet-tunnel</key>
        <string>Helix VPN Extension App Store</string>
    </dict>
</dict>
</plist>
```

7.3 HarmonyOS Build Configuration

```bash
# Prerequisites
# 1. Install DevEco Studio
# 2. Configure HarmonyOS SDK (API 12+)
# 3. Set up signing certificates from Huawei Developer

# Build HAP package
flutter build hap --release

# Output:
# build/ohos/outputs/default/entry-default-signed.hap

# For debug build
flutter run --debug
```

```bash
# Install on device (requires hdc tool)
hdc install -r build/ohos/outputs/default/entry-default-signed.hap
```

7.4 CI/CD Pipeline for Mobile Builds

```yaml
# .github/workflows/mobile-build.yml
name: Mobile Build Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  build-android:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Flutter
        uses: subosito/flutter-action@v2
        with:
          # Mainline Flutter (3.29+), NOT the HarmonyOS community
`-ohos`
          # fork used by the build-harmonyos job below — see §1.1.
          flutter-version: '3.29.0'

      - name: Setup Java
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'

      - name: Setup Rust
        uses: dtolnay/rust-action@stable

      - name: Install cargo-ndk
        run: cargo install cargo-ndk

      - name: Add Android Rust targets
        run: |
          rustup target add aarch64-linux-android
          rustup target add armv7-linux-androideabi
          rustup target add x86_64-linux-android

      - name: Build Rust core
        run: |
          cd helix-core
```

```yaml
          cargo ndk -t armeabi-v7a -t arm64-v8a -t x86_64 -o ../
        android/app/src/main/jniLibs build --release

      - name: Get Flutter dependencies
        run: flutter pub get

      - name: Build Play Store AAB
        run: flutter build appbundle --release --flavor playStore

      - name: Build F-Droid APKs
        run: flutter build apk --release --flavor fdroid --split-
        per-abi

      - name: Upload artifacts
        uses: actions/upload-artifact@v4
        with:
          name: android-builds
          path: |
            build/app/outputs/bundle/playStoreRelease/
            build/app/outputs/apk/fdroid/release/

  build-ios:
    runs-on: macos-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Flutter
        uses: subosito/flutter-action@v2
        with:
          # Mainline Flutter (3.29+) — same rationale as build-
        android above.
          flutter-version: '3.29.0'

      - name: Setup Rust
        uses: dtolnay/rust-action@stable

      - name: Add iOS Rust targets
        run: |
          rustup target add aarch64-apple-ios

      - name: Build Rust core
        run: |
          cd helix-core
          cargo build --target aarch64-apple-ios --release

      - name: Install dependencies
        run: flutter pub get

      - name: Build iOS
        run: flutter build ios --release --no-codesign

      - name: Build IPA
```

```yaml
        run: flutter build ipa --release --no-codesign

      - name: Upload artifacts
        uses: actions/upload-artifact@v4
        with:
          name: ios-build
          path: build/ios/ipa/

  build-harmonyos:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup Flutter (HarmonyOS)
        uses: subosito/flutter-action@v2
        with:
          flutter-version: '3.22.0-ohos'
          channel: 'stable'

      - name: Setup HarmonyOS SDK
        run: |
          # Download and configure HarmonyOS SDK
          wget https://contentcenter-drcn.dbankcdn.com/harmonyos/
        sdk/HarmonyOS-SDK-API12.tar.gz
          tar -xzf HarmonyOS-SDK-API12.tar.gz -C $HOME/
          echo "OHOS_SDK=$HOME/HarmonyOS-SDK" >> $GITHUB_ENV

      - name: Setup Rust
        uses: dtolnay/rust-action@stable

      - name: Build Rust core for HarmonyOS
        run: |
          cd helix-core
          cargo build --target aarch64-linux-ohos --release

      - name: Build HAP
        run: flutter build hap --release

      - name: Upload artifacts
        uses: actions/upload-artifact@v4
        with:
          name: harmonyos-build
          path: build/ohos/outputs/
```

7.5 Code Signing and Certificates

Android:

Certificate Type	Purpose	Location
Debug keystore	Development builds	

Certificate Type	Purpose	Location
		`~/.android/debug.keystore`
Release keystore	Play Store / distribution	Secure CI secret
Upload key	Google Play signing	Managed by Google Play

```
# Generate release keystore
keytool -genkey -v \
  -keystore helix-release.keystore \
  -alias helix \
  -keyalg RSA \
  -keysize 2048 \
  -validity 10000

# Extract certificate fingerprint for API key registration
keytool -list -v -keystore helix-release.keystore -alias helix
```

iOS:

Certificate	Purpose	Type
Apple Development	Development builds	Development
Apple Distribution	App Store / TestFlight	Distribution
Network Extension entitlement	VPN capability	Self-serve

```
# Certificates managed through Apple Developer portal:
# https://developer.apple.com/account/resources/certificates/list

# Provisioning profiles:
# 1. Helix VPN App Development (development cert)
# 2. Helix VPN App Store (distribution cert)
# 3. Helix VPN Extension Development
# 4. Helix VPN Extension App Store
```

HarmonyOS:

Certificate	Source
App signing certificate	Huawei AppGallery Connect
App signing private key (.p12)	Generated in DevEco Studio
Profile file (.p7b)	AppGallery Connect

```
# Generate signing materials in DevEco Studio:
# Build -> Generate Key and CSR
# Upload CSR to AppGallery Connect
```

```
# Download certificate and profile
# Configure signing in build-profile.json5
```

7.6 Distribution Channels Summary

Platform	Channel	Format	Requirements
Android	Google Play	AAB	Developer account ($25 one-time), privacy policy
Android	F-Droid	APK	FOSS license, reproducible builds, F-Droid metadata
Android	Direct/APK	APK	Self-hosted, user enables "Unknown sources"
iOS	App Store	IPA	Developer Program ($99/year), app review
iOS	TestFlight	IPA	App Store Connect, up to 10,000 external testers
HarmonyOS	AppGallery	HAP	Huawei Developer account, security review

---

Appendix A: File Structure

```
helix-vpn-mobile/
├── android/                           # Android platform code
│   ├── app/
│   │   ├── build.gradle
│   │   ├── proguard-rules.pro
│   │   └── src/
│   │       ├── main/
│   │       │   ├── AndroidManifest.xml
│   │       │   ├── kotlin/
│   │       │   │   └── com/helix/vpn/
│   │       │   │       ├── MainActivity.kt
│   │       │   │       ├── vpn/
│   │       │   │       │   ├── HelixVpnService.kt
│   │       │   │       │   ├── VpnNotificationHelper.kt
│   │       │   │       │   └── HelixVpnTileService.kt
│   │       │   │       ├── model/
│   │       │   │       │   ├── VpnConfig.kt
│   │       │   │       │   └── ConnectionStats.kt
│   │       │   │       └── rust/
│   │       │   │           └── HelixRustBridge.kt
│   │       │   └── cpp/                # CMake NDK bridge (if needed)
│   │       ├── playStore/
│   │       └── fdroid/
│   └── build.gradle
```

```
│
├── ios/                              # iOS platform code
│   ├── Runner/
│   │   ├── AppDelegate.swift
│   │   ├── Info.plist
│   │   └── Runner.entitlements
│   ├── PacketTunnelProvider/
│   │   ├── PacketTunnelProvider.swift
│   │   ├── Info.plist
│   │   └── PacketTunnelProvider.entitlements
│   ├── HelixVpnWidget/
│   │   └── HelixVpnWidget.swift
│   ├── ShareExtension/
│   │   └── ShareViewController.swift
│   └── Runner.xcodeproj/
│
├── ohos/                             # HarmonyOS platform code
│   ├── entry/
│   │   ├── src/main/
│   │   │   ├── ets/
│   │   │   │   ├── entryability/
│   │   │   │   │   └── EntryAbility.ets
│   │   │   │   ├── vpnability/
│   │   │   │   │   └── HelixVpnAbility.ets
│   │   │   │   ├── rust/
│   │   │   │   │   └── HelixRustBridge.ets
│   │   │   │   └── widget/
│   │   │   │       └── HelixWidgetAbility.ets
│   │   │   └── cpp/
│   │   │       ├── napi_helix_bridge.cpp
│   │   │       └── CMakeLists.txt
│   │   └── module.json5
│   └── build-profile.json5
│
├── lib/                              # Flutter Dart code
│   ├── main.dart
│   ├── app.dart
│   ├── theme/
│   │   └── helix_theme.dart
│   ├── models/
│   │   ├── connection_stats.dart
│   │   ├── server.dart
│   │   └── app_settings.dart
│   ├── providers/
│   │   ├── connection_provider.dart
│   │   ├── server_provider.dart
│   │   └── settings_provider.dart
│   ├── services/
│   │   ├── vpn_service.dart
│   │   ├── biometric_service.dart
│   │   ├── haptic_service.dart
│   │   ├── notification_service.dart
```

```
│   │   ├── auto_connect_service.dart
│   │   └── deep_link_service.dart
│   ├── ui/
│   │   ├── screens/
│   │   │   ├── connection_screen.dart
│   │   │   ├── server_selection_screen.dart
│   │   │   ├── settings_screen.dart
│   │   │   ├── split_tunneling_screen.dart
│   │   │   └── account_screen.dart
│   │   ├── widgets/
│   │   │   ├── connection_button.dart
│   │   │   ├── connection_stats_panel.dart
│   │   │   ├── country_flag.dart
│   │   │   ├── server_list_tile.dart
│   │   │   └── speed_chart.dart
│   │   └── platform/
│   │       └── platform_adapters.dart
│   └── generated/                    # flutter_rust_bridge
generated
│
├── helix-core/                       # Shared Rust core
│   ├── src/
│   │   ├── lib.rs
│   │   ├── android/
│   │   ├── ios/
│   │   ├── harmonyos/
│   │   ├── protocol/
│   │   ├── crypto/
│   │   └── state/
│   └── Cargo.toml
│
├── pubspec.yaml
├── build.gradle
└── README.md
```

Appendix B: Platform Comparison Matrix

Feature	Android	iOS	HarmonyOS
VPN API	VpnService	NEPacketTunnelProvider	VpnExtensionAbilit
Min OS	API 26 (8.0) — product-supported minimum	iOS 15	API 12 (NEXT)
Flutter	3.29+ (Impeller)	3.29+ (Impeller/Metal)	3.22-ohos (community embedding, pinned

Feature	Android	iOS	HarmonyOS
Code Reuse	72%	72%	70%
Bundle Size Target	< 25 MB	< 25 MB	< 25 MB
Idle RAM Target	< 120 MB	< 120 MB	< 120 MB
Background	Foreground service	System extension	ExtensionAbility
Memory Limit	No hard OS cap on the foreground service itself; ~200MB+ is a practical working budget, distinct from the <120MB whole-app idle target above	~15 MB hard cap on the Network Extension process (`MVP2_ARCHITECTURE.md` §4)	Not officially published by Huawei; engineering target is the same <120MB whole-ap idle budget until measured
Split Tunneling	App + Route-based	Route-based only	App-based
Always-On VPN	Yes (system)	On-Demand rules	Limited
Kill Switch	Lockdown mode	Limited	isBlocking flag
Quick Toggle	QS Tile	Control Center	Form card
Biometrics	BiometricPrompt	Face ID / Touch ID	System biometric
Credential Store	Keystore	Keychain	HUKS
Notifications	FCM	APNs	HMS Push
Distribution	Play/F-Droid/Direct	App Store/TestFlight	AppGallery

---

This specification is a living document. As the MVP2 project evolves, implementation details may be refined based on prototyping results and platform API changes.

End of Document